

✧ Aircraft Maintenance Dataset EDA

Course: Applied Machine Learning Group 11

✧ 1. Setup & Data Loading

```
# Install once if missing – uncomment and run, then re-comment
# !pip install pandas numpy matplotlib seaborn scikit-learn scipy statsmodels \
#      dask[dataframe] pyarrow umap-learn missingno tqdm -q
```

```
import os, sys, gc, warnings, math
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.ticker as mtick
import seaborn as sns
from collections import Counter
from itertools import combinations

# Stats
from scipy import stats
from scipy.stats import skew, kurtosis, ks_2samp, mannwhitneyu, spearmanr

# ML utilities
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler, RobustScaler
from sklearn.feature_selection import mutual_info_classif
from sklearn.cluster import AgglomerativeClustering

# Missing-value visualisation
try:
    import missingno as msno
except ImportError:
    msno = None

# UMAP (optional – falls back to t-SNE if missing)
try:
    import umap
    HAS_UMAP = True
except ImportError:
    HAS_UMAP = False
    from sklearn.manifold import TSNE

warnings.filterwarnings('ignore')

# Plot style
plt.rcParams.update({
    'figure.figsize': (14, 5),
    'figure.dpi': 100,
    'axes.grid': True,
    'grid.alpha': 0.25,
    'axes.spines.top': False,
    'axes.spines.right': False,
    'font.size': 10,
})
sns.set_palette('Set2')

# Reproducibility
RNG = np.random.default_rng(42)

print('Libraries loaded. UMAP available:', HAS_UMAP, ' | missingno available:', msno is not None)
```

```
Libraries loaded. UMAP available: True | missingno available: True
```

```
# DATA SOURCE SELECTOR
DATA_SOURCE = 'full' # 'simulate' / '2days' / 'full'
FULL_DATA_DIR = '/kaggle/input/datasets/hooong/aviation-maintenance-dataset-from-the-ngafid/all_flights/all_flights'
TWO_DAYS_DIR = '/kaggle/input/datasets/hooong/aviation-maintenance-dataset-from-the-ngafid/2days/2days'
channel_names = [
    'volt1', 'volt2', 'amp1', 'amp2', 'FQtyL', 'FQtyR',
    'E1 FFlow', 'E1 OilT', 'E1 OilP', 'E1 RPM',
    'E1 CHT1', 'E1 CHT2', 'E1 CHT3', 'E1 CHT4',
    'E1 EGT1', 'E1 EGT2', 'E1 EGT3', 'E1 EGT4',
    'OAT', 'IAS', 'VSpd', 'NormAc', 'AltMSL'
]

# Channel groupings
CHANNEL_GROUPS = {
    'Electrical': ['volt1', 'volt2', 'amp1', 'amp2'],
    'Fuel': ['FQtyL', 'FQtyR', 'E1 FFlow'],
    'Engine - Oil': ['E1 OilT', 'E1 OilP'],
    'Engine - Power': ['E1 RPM'],
    'Engine - CHT': ['E1 CHT1', 'E1 CHT2', 'E1 CHT3', 'E1 CHT4'],
    'Engine - EGT': ['E1 EGT1', 'E1 EGT2', 'E1 EGT3', 'E1 EGT4'],
    'Environment': ['OAT'],
    'Flight Performance': ['IAS', 'VSpd', 'NormAc', 'AltMSL'],
}

# Expected physical ranges for a Cessna 172 (Licensing 0-230 / 0-260)
```

```

# Source: Cessna 172 POH typical operating ranges. These are *typical*, not absolute hard limits.
PHYSICAL_RANGES = {
    'volt1': (24, 30), # 28V electrical bus, alternator on
    'volt2': (24, 30),
    'amp1': (-30, 60), # alternator output current
    'amp2': (-30, 60),
    'FQtyL': (0, 30), # left tank, US gallons
    'FQtyR': (0, 30),
    'E1 FFlow': (0, 20), # fuel flow, GPH
    'E1 OilT': (75, 245), # °F (idle warm-up to redline)
    'E1 OilP': (20, 115), # PSI
    'E1 RPM': (0, 2700),
    'E1 CHT1': (150, 500), # °F - cylinder head temp
    'E1 CHT2': (150, 500),
    'E1 CHT3': (150, 500),
    'E1 CHT4': (150, 500),
    'E1 EGT1': (800, 1650), # °F - exhaust gas temp
    'E1 EGT2': (800, 1650),
    'E1 EGT3': (800, 1650),
    'E1 EGT4': (800, 1650),
    'OAT': (-40, 50), # °C - outside air temp
    'IAS': (0, 200), # kt - indicated airspeed
    'VSpd': (-3000, 3000), # fpm - vertical speed
    'NormAc': (-2, 4), # g - normal acceleration
    'AltMSL': (-500, 20000), # ft
}

flight_header_df = None
flight_data_df = None
flight_data_array = None

print(f'DATA_SOURCE = {DATA_SOURCE!r}')

```

```
DATA_SOURCE = 'full'
```

```

# Load data
if DATA_SOURCE == 'full':
    import dask.dataframe as dd
    flight_data_df = dd.read_parquet(os.path.join(FULL_DATA_DIR, 'one_parq'))
    flight_header_df = pd.read_csv(
        os.path.join(FULL_DATA_DIR, 'flight_header.csv'), index_col='Master Index'
    )
    print('Full dataset loaded via Dask.')
    print(f' flight_header_df : {flight_header_df.shape}')
    print(f' flight_data_df : {flight_data_df.shape} (lazy)')
    print(f' parquet partitions: {flight_data_df.npartitions}')

elif DATA_SOURCE == '2days':
    from compress_pickle import load as cload
    flight_header_df = pd.read_csv(
        os.path.join(TWODAYS_DIR, 'flight_header.csv'), index_col='Master Index'
    )
    flight_data_array = cload(os.path.join(TWODAYS_DIR, 'flight_data.pkl'))
    print('2-day sample loaded.')
    print(f' flight_header_df : {flight_header_df.shape}')
    print(f' flight_data_array: {len(flight_data_array)} flights')

elif DATA_SOURCE == 'simulate':
    # Synthetic schema only - not real values
    n_flights = 600
    MAINT = ['intake gasket leak/damage', 'oil change', 'propeller repair', 'magneto replacement',
            'exhaust leak', 'spark plug replacement', 'fuel injector cleaning', 'carburetor overhaul']
    flight_header_df = pd.DataFrame({
        'before_after': RNG.choice(['before', 'after', 'same'], n_flights, p=[0.42, 0.36, 0.22]),
        'date_diff': RNG.integers(-30, 15, n_flights),
        'flight_length': RNG.integers(200, 8000, n_flights).astype(float),
        'label': RNG.choice(MAINT, n_flights),
        'hierarchy': RNG.choice([np.nan, 'engine', 'airframe', 'propeller'], n_flights),
        'number_flights_before': RNG.integers(-1, 20, n_flights),
    }, index=pd.RangeIndex(1, n_flights+1, name='Master Index'))
    flight_data_array = {
        idx: RNG.standard_normal((int(flight_header_df.loc[idx, 'flight_length']), 23)).astype(np.float32)
        for idx in flight_header_df.index
    }
    print('Synthetic dataset ready.')

else:
    raise ValueError(f'Unknown DATA_SOURCE {DATA_SOURCE!r}')

print(f' flight_header_df memory: {flight_header_df.memory_usage(deep=True).sum()//1e6:.1f} MB')

```

```

-----
FileNotFoundError                                Traceback (most recent call last)
/usr/local/lib/python3.12/dist-packages/dask/backends.py in wrapper(*args, **kwargs)
    139         try:
--> 140             return func(*args, **kwargs)
    141         except Exception as e:

-----
      14 frames
FileNotFoundError: /kaggle/input/datasets/hooong/aviation-maintenance-dataset-from-the-ngafid/all_flights/all_flights/one_parq

The above exception was the direct cause of the following exception:

FileNotFoundError                                Traceback (most recent call last)
/usr/local/lib/python3.12/dist-packages/dask/backends.py in wrapper(*args, **kwargs)
    149         raise e
    150     else:
--> 151         raise exc from e
    152
    153     wrapper.__name__ = dispatch_name

FileNotFoundError: An error occurred while calling the read_parquet method registered to the pandas backend.
Original Message: /kaggle/input/datasets/hooong/aviation-maintenance-dataset-from-the-ngafid/all_flights/all_flights/one_parq

```

Next steps: [Explain error](#)

2. Flight Header Overview

The header CSV holds one row per flight with the label columns and aggregate metadata. Before doing anything else we need to know its schema, dtypes, memory footprint, and which columns are usable.

```

# Shape, dtype, memory, null inventory – all in one place
print('=' * 70)
print(f'flight_header_df :: {flight_header_df.shape[0]:,} rows x {flight_header_df.shape[1]} cols')
print(f'Memory :: {flight_header_df.memory_usage(deep=True).sum()/1e6:.2f} MB')
print('=' * 70)

inv = pd.DataFrame({
    'dtype':      flight_header_df.dtypes.astype(str),
    'non_null':   flight_header_df.notna().sum(),
    'null':       flight_header_df.isna().sum(),
    'null_pct':   flight_header_df.isna().mean() * 100,
    'unique':     flight_header_df.nunique(dropna=True),
    'sample':     [str(flight_header_df[c].dropna().iloc[0]) if flight_header_df[c].notna().any() else '-'
                   for c in flight_header_df.columns],
})
inv['null_pct'] = inv['null_pct'].round(2)
inv.sort_values('null_pct', ascending=False)

```

```

=====
flight_header_df :: 28,935 rows x 6 cols
Memory          :: 5.90 MB
=====

```

	dtype	non_null	null	null_pct	unique	sample
hierarchy	object	10581	18354	63.43	4	engine
before_after	object	28935	0	0.00	3	before
date_diff	int64	28935	0	0.00	84	-1
flight_length	float64	28935	0	0.00	5519	4723.0
label	object	28935	0	0.00	36	intake gasket leak/damage
number_flights_before	int64	28935	0	0.00	6	0

```

# Inspect the parquet (sensor data) structure on the Dask side
if flight_data_df is not None:
    print(f'Sensor parquet columns : {list(flight_data_df.columns)}')
    print(f'Index name           : {flight_data_df.index.name}')
    print(f'Partitions            : {flight_data_df.npartitions}')
    # One partition row-count gives us a quick sense of rows per partition without computing all
    sample_part_rows = flight_data_df.get_partition(0).shape[0].compute()
    print(f'Rows in partition 0    : {sample_part_rows:,}')
elif flight_data_array is not None:
    lengths = [arr.shape[0] for arr in flight_data_array.values()]
    print(f'Sensor arrays           : {len(flight_data_array)}')
    print(f'Total timesteps         : {sum(lengths):,}')
    print(f'Channels per flight         : {next(iter(flight_data_array.values())).shape[1]}')

Sensor parquet columns : ['volt1', 'volt2', 'amp1', 'amp2', 'FQtyL', 'FQtyR', 'E1 FFlow', 'E1 OilT', 'E1 OilP', 'E1 RPM', 'E1 CHT1', 'E1 CHT2', 'E1 CHT3', 'E1 CHT4']
Index name             : Master Index
Partitions              : 401
Rows in partition 0     : 282,875

```

3. Target Variables — Deep Dive

The slide deck identifies TWO target outputs: **label** (36-class fine-grained maintenance type) and **hierarchy** (coarser grouping). The teammate's EDA only looked at `before_after`. Here we examine **all four label-related columns** and quantify how usable each one is.

The four header columns relevant to the supervised task are:

Column	Meaning
before_after	Was this flight before, after, or on the day of a maintenance event?
label	The 36-class maintenance-type label (the fine-grained target)
hierarchy	Coarser grouping of label (often missing)
date_diff	Days between this flight and the maintenance event (negative = before)
number_flights_before	Count of preceding flights linked to the same event

```
# Side-by-side summary of every label-related column
target_cols = [c for c in ['before_after', 'label', 'hierarchy', 'date_diff', 'number_flights_before']
                if c in flight_header_df.columns]

summary = []
for col in target_cols:
    s = flight_header_df[col]
    summary.append({
        'column': col,
        'dtype': str(s.dtype),
        'non_null': s.notna().sum(),
        'null_pct': round(s.isna().mean() * 100, 2),
        'unique': s.nunique(dropna=True),
        'top_value': s.value_counts(dropna=True).index[0] if s.notna().any() else '-',
        'top_pct': round(s.value_counts(normalize=True, dropna=True).iloc[0]*100, 2)
                    if s.notna().any() else 0,
    })
pd.DataFrame(summary)
```

	column	dtype	non_null	null_pct	unique	top_value	top_pct
0	before_after	object	28935	0.00	3	before	41.96
1	label	object	28935	0.00	36	intake gasket leak/damage	35.04
2	hierarchy	object	10581	63.43	4	affle	51.52
3	date_diff	int64	28935	0.00	84	0	29.31
4	number_flights_before	int64	28935	0.00	6	-1	58.04

3.1 Binaryish target: before_after

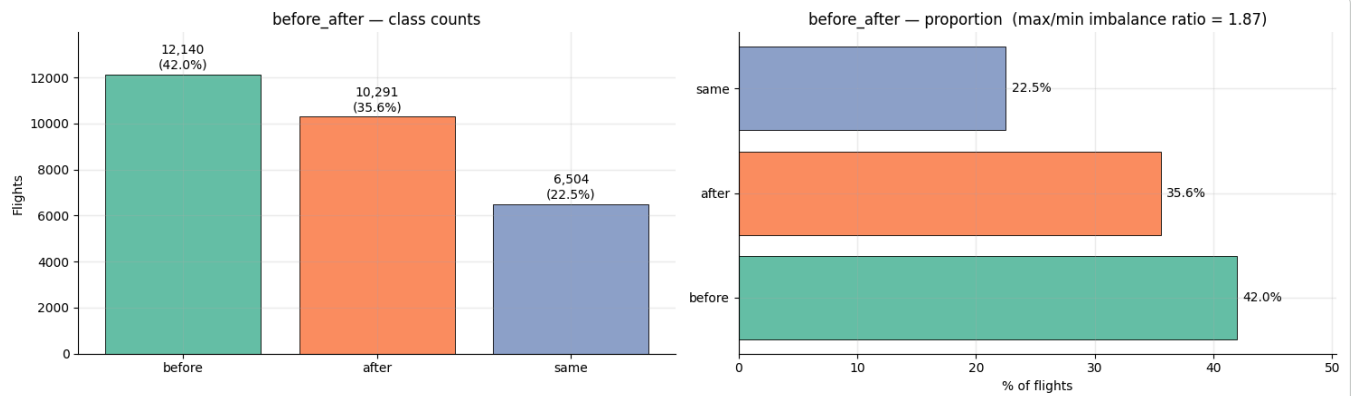
```
ba_counts = flight_header_df['before_after'].value_counts()
ba_pct = ba_counts / ba_counts.sum() * 100

fig, axes = plt.subplots(1, 2, figsize=(15, 4.5))

# Counts (bar) with values on top
bars = axes[0].bar(ba_counts.index, ba_counts.values, color=sns.color_palette('Set2', 3),
                  edgecolor='black', linewidth=0.6)
for bar, v, p in zip(bars, ba_counts.values, ba_pct.values):
    axes[0].text(bar.get_x() + bar.get_width()/2, v + ba_counts.max()*0.01,
                 f'{v:,}\n({p:.1f}%)', ha='center', va='bottom', fontsize=10)
axes[0].set_title('before_after - class counts')
axes[0].set_ylabel('Flights')
axes[0].set_ylim(0, ba_counts.max() * 1.15)

# Class-imbalance ratio
imb_ratio = ba_counts.max() / ba_counts.min()
axes[1].barh(ba_counts.index, ba_pct.values, color=sns.color_palette('Set2', 3),
             edgecolor='black', linewidth=0.6)
for i, p in enumerate(ba_pct.values):
    axes[1].text(p + 0.5, i, f'{p:.1f}%', va='center', fontsize=10)
axes[1].set_title(f'before_after - proportion (max/min imbalance ratio = {imb_ratio:.2f})')
axes[1].set_xlabel('% of flights')
axes[1].set_xlim(0, ba_pct.max() * 1.2)
plt.tight_layout(); plt.show()

print('\nClass distribution:')
print(pd.DataFrame({'count': ba_counts, 'pct': ba_pct.round(2)}))
```



Class distribution:

	count	pct
before_after		
before	12140	41.96
after	10291	35.57
same	6504	22.48

3.2 The 36-class `label` target

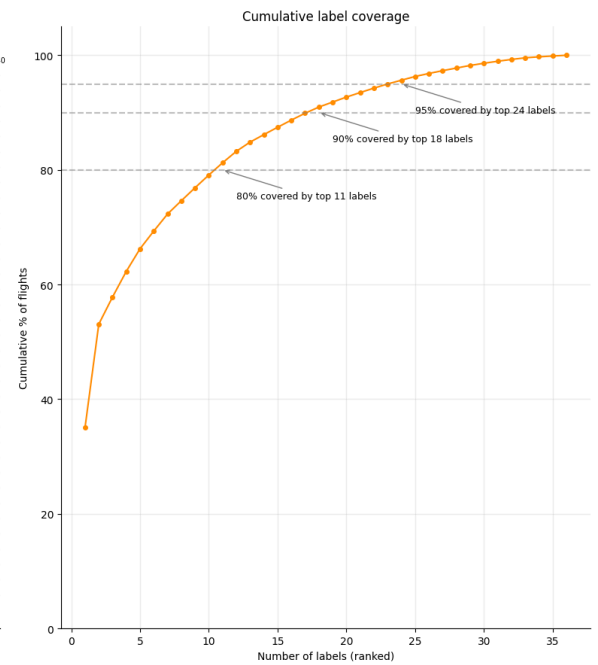
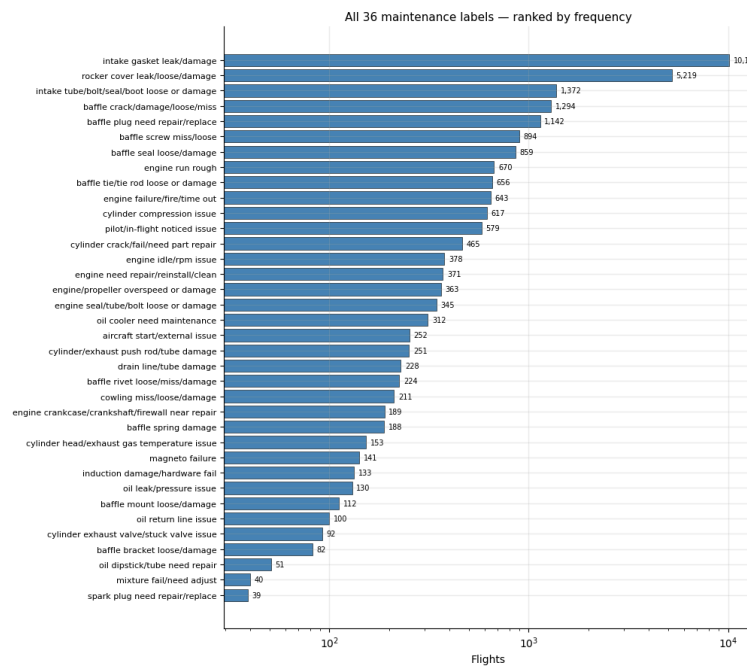
```
label_counts = flight_header_df['label'].value_counts()
label_pct = label_counts / label_counts.sum() * 100

fig, axes = plt.subplots(1, 2, figsize=(18, max(7, len(label_counts)*0.25)))

# (a) Full ranked bar chart
y = np.arange(len(label_counts))
axes[0].barh(y, label_counts.values, color='steelblue', edgecolor='black', linewidth=0.4)
axes[0].set_yticks(y)
axes[0].set_yticklabels(label_counts.index, fontsize=8)
axes[0].invert_yaxis()
axes[0].set_xlabel('Flights')
axes[0].set_title(f'All {len(label_counts)} maintenance labels – ranked by frequency', fontsize=11)
axes[0].set_xscale('log') # log scale because the tail is severe
for i, v in enumerate(label_counts.values):
    axes[0].text(v * 1.05, i, f'{v:,}', va='center', fontsize=7)

# (b) Cumulative coverage
cum = label_counts.cumsum() / label_counts.sum() * 100
axes[1].plot(np.arange(1, len(cum)+1), cum.values, marker='o', markersize=4, color='darkorange')
for thresh in [80, 90, 95]:
    n_needed = (cum < thresh).sum() + 1
    axes[1].axhline(thresh, ls='--', color='gray', alpha=0.5)
    axes[1].annotate(f'{thresh}% covered by top {n_needed} labels',
                    xy=(n_needed, thresh), xytext=(n_needed+1, thresh-5),
                    fontsize=9, arrowprops=dict(arrowstyle='->', color='gray'))
axes[1].set_xlabel('Number of labels (ranked)')
axes[1].set_ylabel('Cumulative % of flights')
axes[1].set_title('Cumulative label coverage')
axes[1].set_ylim(0, 105)
plt.tight_layout(); plt.show()

# Printed tail summary
print(f'Labels with < 10 flights : {(label_counts < 10).sum()}')
print(f'Labels with < 50 flights : {(label_counts < 50).sum()}')
print(f'Labels with < 100 flights : {(label_counts < 100).sum()}')
print(f'Top label "{label_counts.index[0]}" alone covers {label_pct.iloc[0]:.1f}% of flights')
```

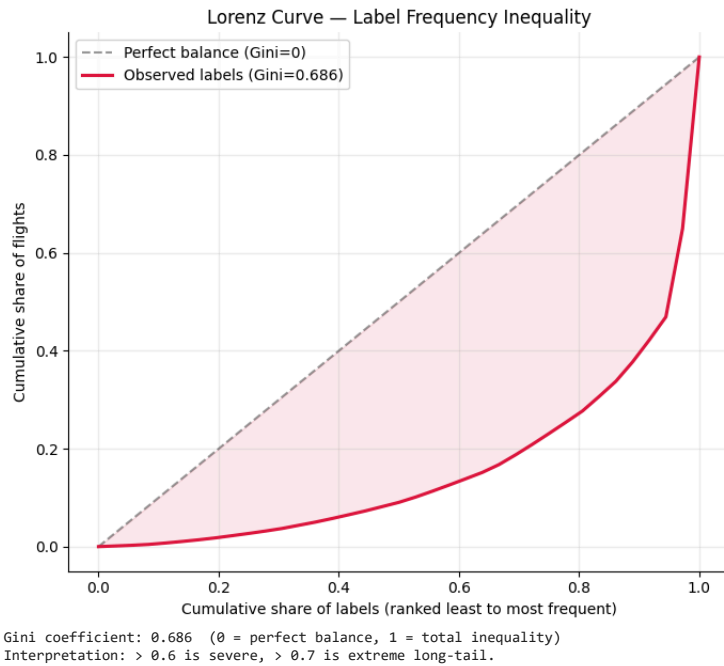


Labels with < 10 flights : 0
 Labels with < 50 flights : 2
 Labels with < 100 flights : 5
 Top label "intake gasket leak/damage" alone covers 35.0% of flights

```
# Lorenz curve + Gini coefficient
sorted_counts = np.sort(label_counts.values)
n = len(sorted_counts)
cum_share = np.cumsum(sorted_counts) / sorted_counts.sum()
pop_share = np.arange(1, n+1) / n
# Gini = 1 - 2 * area under Lorenz
gini = 1 - 2 * np.trapz(cum_share, pop_share)

fig, ax = plt.subplots(figsize=(7, 6))
ax.plot([0, 1], [0, 1], 'k--', alpha=0.4, label='Perfect balance (Gini=0)')
ax.plot(np.insert(pop_share, 0, 0), np.insert(cum_share, 0, 0),
        color='crimson', linewidth=2.2, label=f'Observed labels (Gini={gini:.3f})')
ax.fill_between(np.insert(pop_share, 0, 0), np.insert(cum_share, 0, 0),
               np.insert(pop_share, 0, 0), color='crimson', alpha=0.1)
ax.set_xlabel('Cumulative share of labels (ranked least to most frequent)')
ax.set_ylabel('Cumulative share of flights')
ax.set_title('Lorenz Curve - Label Frequency Inequality')
ax.legend(loc='upper left')
plt.tight_layout(); plt.show()

print(f'Gini coefficient: {gini:.3f} (0 = perfect balance, 1 = total inequality)')
print('Interpretation: > 0.6 is severe, > 0.7 is extreme long-tail.')
```



3.3 hierarchy column

```
h_missing = flight_header_df['hierarchy'].isna().mean() * 100
print(f'hierarchy null rate: {h_missing:.2f}%')
print(f'Distinct hierarchy values: {flight_header_df["hierarchy"].nunique()}\n')

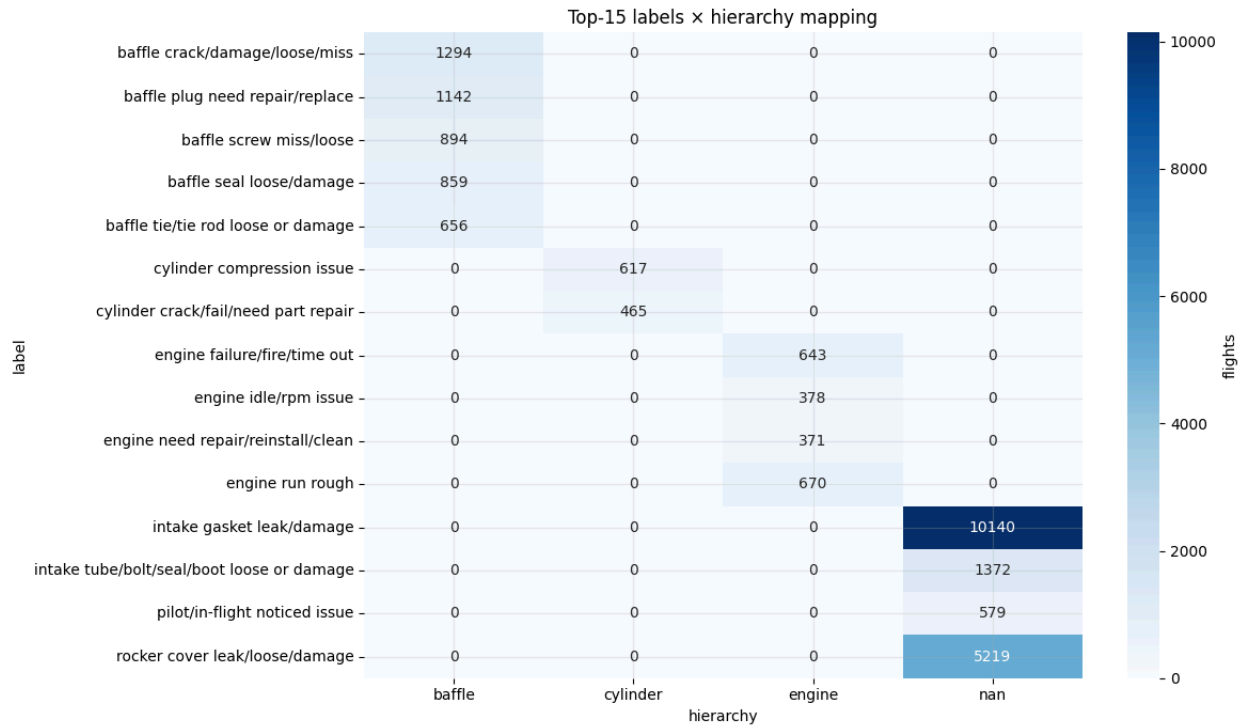
if h_missing < 100:
    # Hierarchy distribution
    h_counts = flight_header_df['hierarchy'].value_counts(dropna=False)
    print('Hierarchy distribution (incl. NaN):')
    print(h_counts.to_string())
    print()

    # Label x Hierarchy crosstab - limited to top 15 labels for readability
    top_labels = label_counts.head(15).index
    ct = pd.crosstab(flight_header_df.loc[flight_header_df['label'].isin(top_labels), 'label'],
                    flight_header_df.loc[flight_header_df['label'].isin(top_labels), 'hierarchy'],
                    dropna=False)

    fig, ax = plt.subplots(figsize=(12, 7))
    sns.heatmap(ct, annot=True, fmt='d', cmap='Blues', cbar_kws={'label': 'flights'}, ax=ax)
    ax.set_title('Top-15 labels x hierarchy mapping')
    plt.tight_layout(); plt.show()
```

hierarchy null rate: 63.43%
Distinct hierarchy values: 4

Hierarchy distribution (incl. NaN):
hierarchy
NaN 18354
baffle 5451
engine 2959
cylinder 1578
oil 593



3.4 date_diff

If `date_diff` is the number of days between a flight and its maintenance event (negative = before), its distribution tells us how the labelling window is defined.

```
if 'date_diff' in flight_header_df.columns:
    dd_data = flight_header_df['date_diff'].dropna()
    print(f'date_diff stats:')
    print(dd_data.describe().round(2))

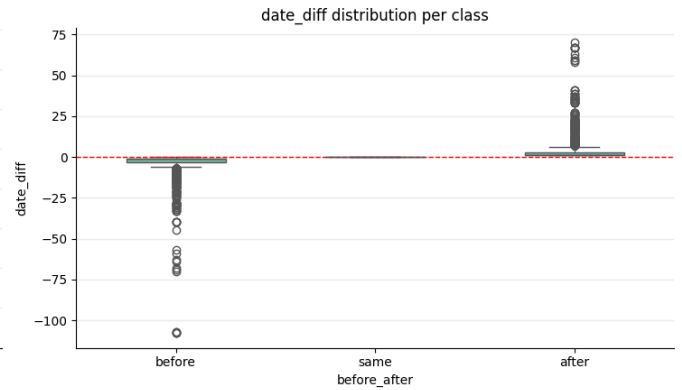
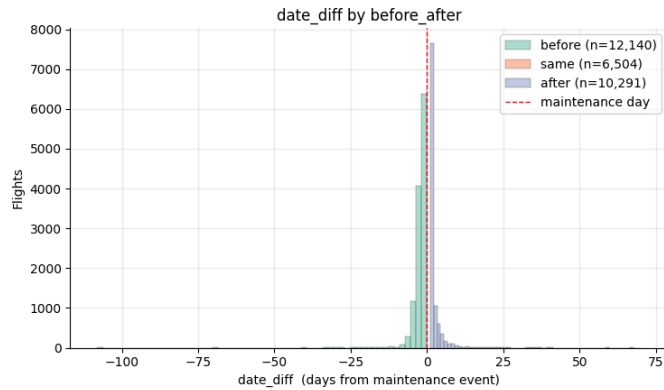
fig, axes = plt.subplots(1, 2, figsize=(15, 4.5))

# (a) Histogram, coloured by before_after
for ba in ['before', 'same', 'after']:
    if ba in flight_header_df['before_after'].unique():
        d = flight_header_df.loc[flight_header_df['before_after']==ba, 'date_diff'].dropna()
        axes[0].hist(d, bins=60, alpha=0.55, label=f'{ba} (n={len(d):,})',
                     edgecolor='black', linewidth=0.3)
axes[0].axvline(0, color='red', ls='--', linewidth=1, label='maintenance day')
axes[0].set_xlabel('date_diff (days from maintenance event)')
axes[0].set_ylabel('Flights')
axes[0].set_title('date_diff by before_after')
axes[0].legend()

# (b) Boxplot per class
sns.boxplot(data=flight_header_df, x='before_after', y='date_diff',
            order=['before', 'same', 'after'], ax=axes[1], width=0.5)
axes[1].axhline(0, color='red', ls='--', linewidth=1)
axes[1].set_title('date_diff distribution per class')
plt.tight_layout(); plt.show()
```



```
date_diff stats:
count    28935.00
mean      -0.03
std        3.72
min       -108.00
25%        -1.00
50%         0.00
75%         1.00
max         70.00
Name: date_diff, dtype: float64
```



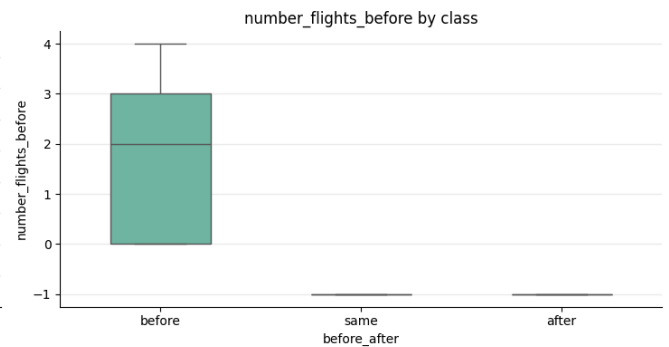
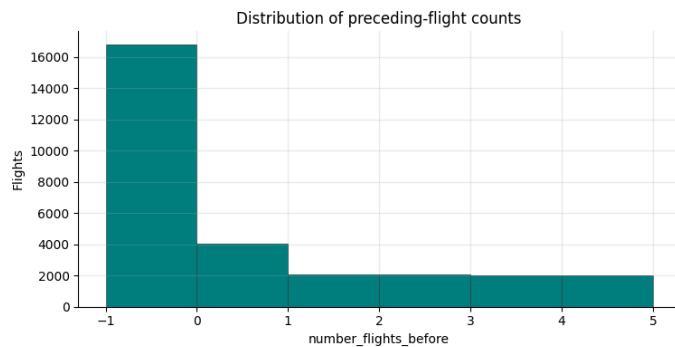
3.5 `number_flights_before` -> how many flights precede each maintenance event?

```
if 'number_flights_before' in flight_header_df.columns:
    nfb = flight_header_df['number_flights_before'].dropna()
    print('number_flights_before stats:')
    print(nfb.describe().round(2), '\n')

    fig, axes = plt.subplots(1, 2, figsize=(15, 4))
    axes[0].hist(nfb, bins=range(int(nfb.min()), int(nfb.max()+2),
                                color='teal', edgecolor='black', linewidth=0.4)
    axes[0].set_xlabel('number_flights_before')
    axes[0].set_ylabel('Flights')
    axes[0].set_title('Distribution of preceding-flight counts')

    sns.boxplot(data=flight_header_df, x='before_after', y='number_flights_before',
                order=['before', 'same', 'after'], ax=axes[1], width=0.5)
    axes[1].set_title('number_flights_before by class')
    plt.tight_layout(); plt.show()
```

```
number_flights_before stats:
count    28935.00
mean      0.12
std        1.63
min       -1.00
25%       -1.00
50%       -1.00
75%        1.00
max         4.00
Name: number_flights_before, dtype: float64
```



4. Flight Length & Temporal Structure

Flight length (number of timesteps) is critical because it determines truncation/padding strategy.

```
f1 = flight_header_df['flight_length'].dropna()
```

```
# Headline stats
print('Flight length (timesteps):')
print(f' count : {len(fl):,}')
print(f' mean : {fl.mean():,.0f}')
print(f' median : {fl.median():,.0f}')
print(f' std : {fl.std():,.0f}')
print(f' min/max : {fl.min():,.0f} / {fl.max():,.0f}')
print(f' skew : {skew(fl):.3f}')
print(f' kurtosis: {kurtosis(fl):.3f}')

# Percentiles for padding-strategy decisions
pcts = [1, 5, 10, 25, 50, 75, 90, 95, 99]
print('\nPercentiles:')
for p in pct:
    print(f' p{p:02d} : {np.percentile(fl, p):,.0f}')
```

```
Flight length (timesteps):
count : 28,935
mean : 3,879
median : 4,402
std : 3,066
min/max : 12 / 30,059
skew : 0.479
kurtosis: 0.310
```

```
Percentiles:
p01 : 14
p05 : 29
p10 : 45
p25 : 419
p50 : 4,402
p75 : 5,771
p90 : 7,375
p95 : 9,212
p99 : 12,108
```

```
fig, axes = plt.subplots(2, 2, figsize=(16, 9))
```

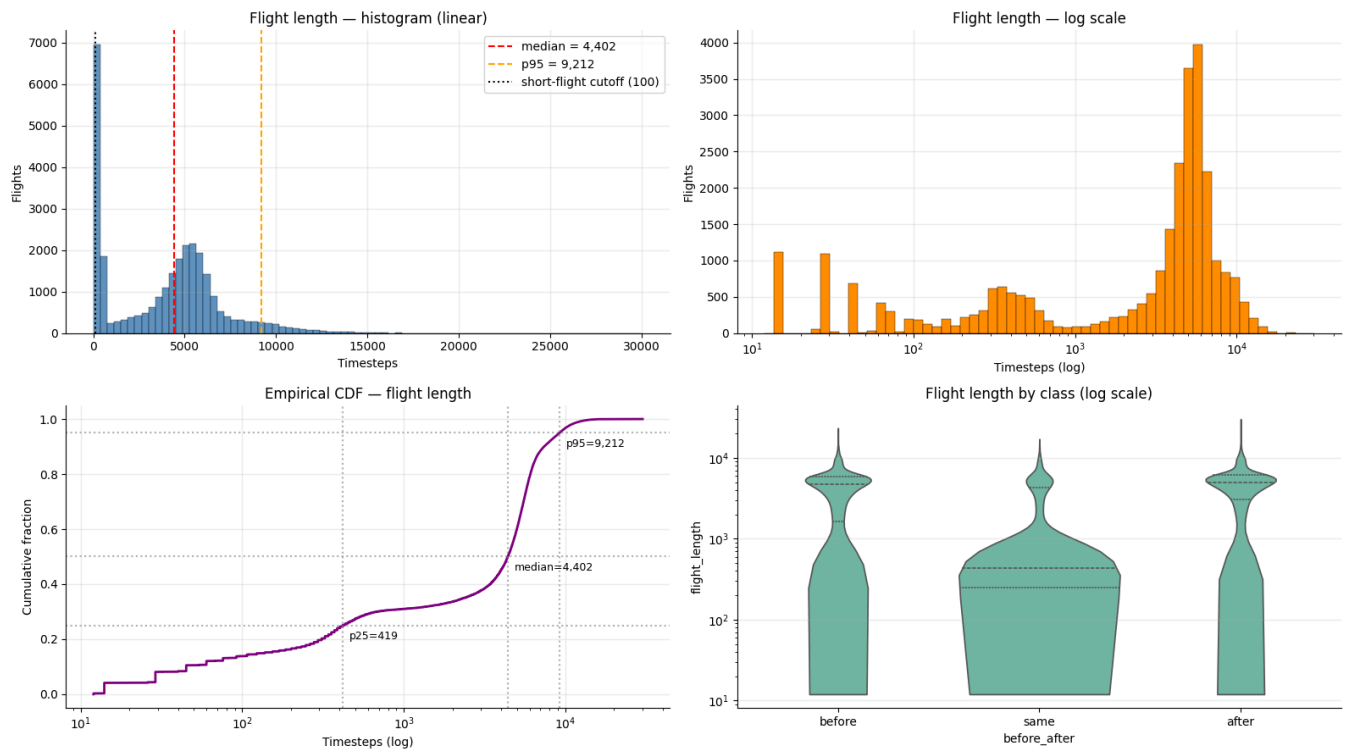
```
# (a) Histogram with key cutoffs
axes[0,0].hist(fl, bins=80, color='steelblue', edgecolor='black', linewidth=0.3, alpha=0.85)
axes[0,0].axvline(fl.median(), color='red', ls='--', label=f'median = {fl.median():,.0f}')
axes[0,0].axvline(np.percentile(fl, 95), color='orange', ls='--', label=f'p95 = {np.percentile(fl,95):,.0f}')
axes[0,0].axvline(100, color='black', ls=':', label='short-flight cutoff (100)')
axes[0,0].set_xlabel('Timesteps'); axes[0,0].set_ylabel('Flights')
axes[0,0].set_title('Flight length - histogram (linear)')
axes[0,0].legend()
```

```
# (b) Log-scale histogram - better for skewed data
axes[0,1].hist(fl, bins=np.logspace(np.log10(max(fl.min(),1)), np.log10(fl.max()), 60),
               color='darkorange', edgecolor='black', linewidth=0.3)
axes[0,1].set_xscale('log')
axes[0,1].set_xlabel('Timesteps (log)'); axes[0,1].set_ylabel('Flights')
axes[0,1].set_title('Flight length - log scale')
```

```
# (c) ECDF with annotations
fl_sorted = np.sort(fl.values)
ecdf = np.arange(1, len(fl_sorted)+1) / len(fl_sorted)
axes[1,0].plot(fl_sorted, ecdf, linewidth=2, color='purple')
for p, label in [(0.25,'p25'), (0.5,'median'), (0.95,'p95')]:
    v = np.percentile(fl, p*100)
    axes[1,0].axhline(p, ls=':', color='gray', alpha=0.6)
    axes[1,0].axvline(v, ls=':', color='gray', alpha=0.6)
    axes[1,0].annotate(f'{label}={v:,.0f}', xy=(v, p), xytext=(v*1.1, p-0.05), fontsize=9)
axes[1,0].set_xscale('log')
axes[1,0].set_xlabel('Timesteps (log)'); axes[1,0].set_ylabel('Cumulative fraction')
axes[1,0].set_title('Empirical CDF - flight length')
```

```
# (d) Violin by class
sns.violinplot(data=flight_header_df, x='before_after', y='flight_length',
               order=['before','same','after'], ax=axes[1,1], cut=0, inner='quartile')
axes[1,1].set_yscale('log')
axes[1,1].set_title('Flight length by class (log scale)')

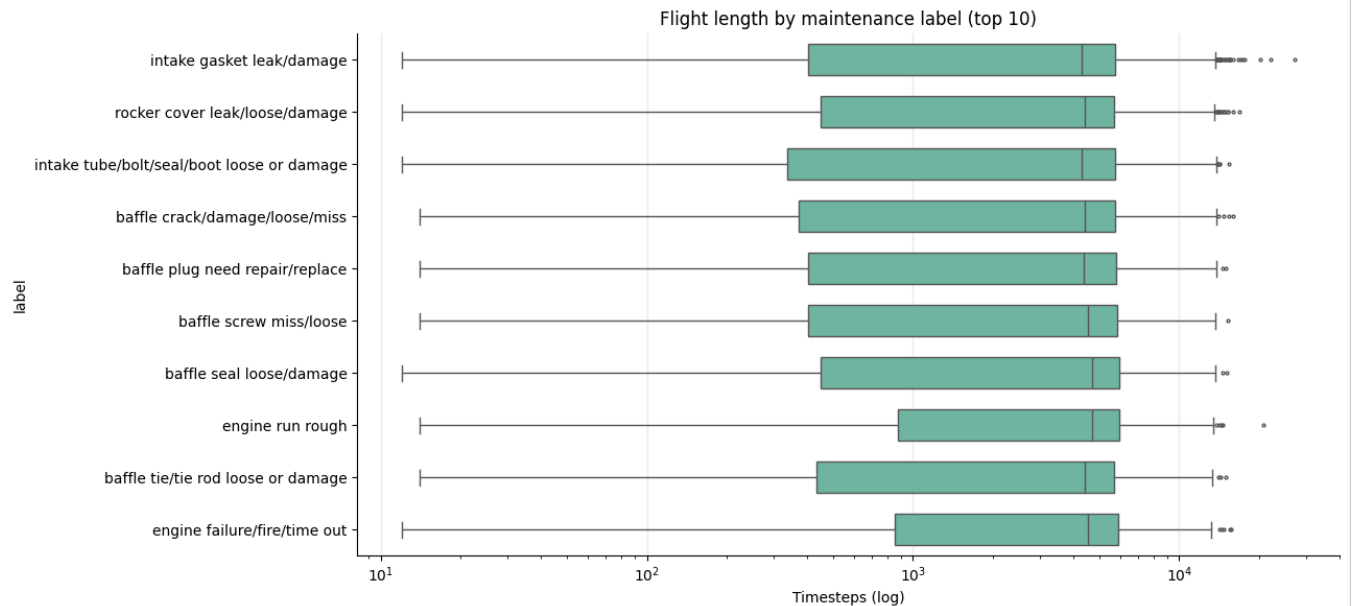
plt.tight_layout(); plt.show()
```



```
# Statistical test: do the three classes have different flight-length distributions?
from scipy.stats import kruskal
groups = [flight_header_df.loc[flight_header_df['before_after']==c, 'flight_length'].dropna()
          for c in ['before', 'same', 'after']]
if len(groups) >= 2:
    H, p = kruskal(*groups)
    print(f'Kruskal-Wallis (flight_length vs before_after): H = {H:.2f} p = {p:.2e}')
    print(f'Interpretation: p {"<" if p<0.05 else ">="} 0.05 → '
          f'{"distributions DIFFER significantly" if p<0.05 else "no significant difference"} across classes.')
```

```
Kruskal-Wallis (flight_length vs before_after): H = 2549.37 p = 0.00e+00
Interpretation: p < 0.05 → distributions DIFFER significantly across classes.
```

```
# Flight length by maintenance label — top 10 labels only
top10 = label_counts.head(10).index
sub = flight_header_df[flight_header_df['label'].isin(top10)]
fig, ax = plt.subplots(figsize=(13, 6))
sns.boxplot(data=sub, y='label', x='flight_length',
            order=top10, ax=ax, fliersize=2, width=0.6)
ax.set_xscale('log')
ax.set_title('Flight length by maintenance label (top 10)')
ax.set_xlabel('Timesteps (log)')
plt.tight_layout(); plt.show()
```



```
# Identify and tally suspiciously short / suspiciously long flights
SHORT_CUT, LONG_CUT = 100, np.percentile(f1, 99)
suspicious = pd.DataFrame({
    'category': ['Very short (< 100 steps)', f'p99 - max (> {LONG_CUT:,} steps)'],
    'count': [(f1 < SHORT_CUT).sum(), (f1 > LONG_CUT).sum()],
    'pct': [(f1 < SHORT_CUT).mean()*100, (f1 > LONG_CUT).mean()*100],
})
print('Outlier flight lengths to inspect / filter:')
print(suspicious.to_string(index=False))
```

```
Outlier flight lengths to inspect / filter:
      category  count  pct
Very short (< 100 steps)  4001 13.827544
p99 - max (> 12,108 steps)  290  1.002246
```

5. Stratified Sampling & Per-Flight Aggregation

```
N_PER_CLASS = 500 # tune for your machine - ~1,500 flights total

# Stratified random sample of master indices
rng_local = np.random.default_rng(42)
sampled_indices = []
for cls in flight_header_df['before_after'].dropna().unique():
    pool = flight_header_df.index[flight_header_df['before_after'] == cls].to_numpy()
    take = min(N_PER_CLASS, len(pool))
    sampled_indices.extend(rng_local.choice(pool, size=take, replace=False))
sampled_indices = np.array(sorted(sampled_indices))
print(f'Sampled flight count: {len(sampled_indices):,}')
print(flight_header_df.loc[sampled_indices, 'before_after'].value_counts())
```

```
Sampled flight count: 1,500
before_after
after    500
same    500
before  500
Name: count, dtype: int64
```

```
from tqdm.auto import tqdm

def aggregate_flight(arr, ch_names):
    """Collapse one flight to {channel}_{stat} features."""
    out = {}
    for j, ch in enumerate(ch_names[:arr.shape[1]]):
        col = arr[:, j].astype(float)
        valid = col[~np.isnan(col)]
        if len(valid) == 0:
            for stat in ['mean', 'std', 'min', 'p05', 'p50', 'p95', 'max', 'range', 'slope', 'last']:
                out[f'{ch}__{stat}'] = np.nan
            continue
        out[f'{ch}__mean'] = valid.mean()
        out[f'{ch}__std'] = valid.std()
        out[f'{ch}__min'] = valid.min()
        out[f'{ch}__p05'] = np.percentile(valid, 5)
        out[f'{ch}__p50'] = np.percentile(valid, 50)
        out[f'{ch}__p95'] = np.percentile(valid, 95)
        out[f'{ch}__max'] = valid.max()
        out[f'{ch}__range'] = valid.max() - valid.min()
        if len(valid) >= 2:
            x = np.arange(len(valid))
            out[f'{ch}__slope'] = np.polyfit(x, valid, 1)[0]
```

```

        else:
            out[f'{ch}__slope'] = 0.0
            out[f'{ch}__last'] = valid[-1]
        return out

agg_records = []
raw_lengths = []

if flight_data_array is not None:
    # 2days / simulate branch - already a dict in memory
    for idx in tqdm(sampled_indices, desc='Aggregating sampled flights'):
        arr = flight_data_array[idx]
        raw_lengths.append(arr.shape[0])
        rec = aggregate_flight(arr, channel_names)
        rec['Master Index'] = idx
        agg_records.append(rec)
else:
    # Dask path - filter inside each partition, then concat
    print('Loading all sampled flights in a single Dask scan...')
    sampled_set = set(sampled_indices)

    def filter_partition(df):
        return df[df.index.isin(sampled_set)]

    sampled_pd = flight_data_df.map_partitions(filter_partition).compute()
    print(f'Loaded {len(sampled_pd):,} rows for {sampled_pd.index.nunique()} flights')

    keep_cols = [c for c in channel_names if c in sampled_pd.columns]
    for idx, group in tqdm(sampled_pd.groupby(level=0), desc='Aggregating sampled flights'):
        if 'timestep' in group.columns:
            group = group.sort_values('timestep')
        arr = group[keep_cols].to_numpy()
        raw_lengths.append(arr.shape[0])
        rec = aggregate_flight(arr, channel_names)
        rec['Master Index'] = idx
        agg_records.append(rec)

flight_features_df = pd.DataFrame(agg_records).set_index('Master Index')
flight_features_df = flight_features_df.join(
    flight_header_df[['before_after', 'label', 'hierarchy', 'flight_length', 'date_diff']],
    how='left'
)
print(f'\nflight_features_df: {flight_features_df.shape}')
flight_features_df.head(3)

```

Loading all sampled flights in a single Dask scan...

Loaded 5,329,809 rows for 1500 flights

Aggregating sampled flights: 0% | 0/1500 [00:00<?, ?it/s]

flight_features_df: (1500, 235)

	volt1__mean	volt1__std	volt1__min	volt1__p05	volt1__p50	volt1__p95	volt1__max	volt1__range	volt1__slope	volt1__last	...	AltMSL__p95	AltMSL__max
Master Index													
58	27.830472	0.762960	24.5	25.5	28.1	28.1	28.2	3.7	-0.000040	28.0	...	4834.05	5043.5
75	24.862069	0.055172	24.8	24.8	24.9	24.9	25.0	0.2	0.000443	25.0	...	NaN	NaN
99	27.820602	0.674275	25.5	25.6	28.0	28.1	28.1	2.6	-0.000626	27.7	...	1611.58	1650.8

3 rows × 235 columns

6. Per-Channel Univariate Analysis

```

# Per-channel descriptive stats - using each flight's mean as the observation
mean_cols = [f'{ch}__mean' for ch in channel_names if f'{ch}__mean' in flight_features_df.columns]

desc = flight_features_df[mean_cols].describe(percentiles=[.05, .25, .5, .75, .95]).T
desc['skew'] = flight_features_df[mean_cols].skew()
desc['kurtosis'] = flight_features_df[mean_cols].kurtosis()
desc['nan_pct'] = flight_features_df[mean_cols].isna().mean() * 100
desc.index = [c.replace('__mean', '') for c in desc.index]
desc = desc[['count', 'mean', 'std', 'min', '5%', '25%', '50%', '75%', '95%', 'max', 'skew', 'kurtosis', 'nan_pct']]
desc.round(3)

```

	count	mean	std	min	5%	25%	50%	75%	95%	max	skew	kurtosis	nan_pct
volt1	1499.0	27.366	1.371	17.023	24.200	27.628	27.894	28.031	28.192	28.873	-2.229	5.439	0.067
volt2	1420.0	27.316	1.443	11.015	24.198	27.610	27.900	28.015	28.132	28.300	-3.127	16.858	5.333
amp1	1499.0	-0.777	5.156	-31.777	-11.925	0.231	0.814	1.407	3.082	35.963	-1.505	4.764	0.067
amp2	1420.0	0.028	0.108	-0.300	-0.105	-0.004	0.001	0.051	0.275	0.638	1.647	4.646	5.333
FQtyL	1500.0	20.031	6.638	0.000	11.974	17.472	19.776	21.840	24.000	54.000	1.871	8.923	0.000
FQtyR	1500.0	21.541	6.669	0.000	13.447	19.479	21.564	23.284	24.000	54.000	1.701	9.401	0.000
E1 FFlow	1499.0	5.073	2.736	0.000	0.000	3.308	6.090	7.134	8.303	10.436	-0.737	-0.667	0.067
E1 OiIT	1498.0	124.158	34.130	21.700	66.587	89.948	134.606	149.322	167.387	185.971	-0.582	-0.843	0.133
E1 OiIP	1499.0	55.069	24.007	-0.959	-0.115	60.421	64.517	67.126	71.017	87.468	-1.765	1.339	0.067
E1 RPM	1499.0	1443.026	707.598	0.000	0.000	1155.674	1745.389	1927.296	2181.414	2456.157	-1.105	-0.008	0.067
E1 CHT1	1494.0	242.809	77.490	7.403	71.730	197.238	271.686	298.209	326.474	379.713	-1.059	0.109	0.400
E1 CHT2	1420.0	241.256	76.357	10.048	72.214	194.894	272.079	295.604	321.086	357.119	-1.071	0.090	5.333
E1 CHT3	1420.0	247.846	78.495	7.807	71.794	202.191	279.211	303.396	329.593	366.120	-1.111	0.176	5.333
E1 CHT4	1420.0	252.022	81.368	9.251	72.469	201.698	284.601	310.721	340.387	380.332	-1.027	0.011	5.333
E1 EGT1	1499.0	1027.080	412.559	2.453	69.576	1069.897	1187.866	1258.074	1341.879	1501.876	-1.694	1.226	0.067
E1 EGT2	1499.0	1035.802	417.384	4.330	69.474	1068.678	1204.272	1270.840	1358.461	1520.274	-1.681	1.199	0.067
E1 EGT3	1499.0	1034.460	414.722	2.236	70.037	1071.716	1198.556	1269.493	1348.486	1492.236	-1.698	1.247	0.067
E1 EGT4	1499.0	1037.229	416.849	3.374	69.301	1074.971	1203.551	1270.806	1359.148	1545.344	-1.689	1.225	0.067
OAT	1499.0	8.167	13.118	-26.000	-16.557	-0.552	10.140	18.526	26.166	40.154	-0.438	-0.563	0.067
IAS	1499.0	42.391	34.819	0.000	0.000	0.000	58.185	71.198	86.821	133.868	-0.195	-1.593	0.067
VSpd	1499.0	-0.509	8.882	-208.480	-3.596	-0.813	0.000	0.774	2.596	69.997	-14.533	300.093	0.067
NormAc	1405.0	0.003	0.007	-0.011	-0.002	-0.000	0.001	0.005	0.016	0.048	2.202	7.277	6.333
AltMSL	1348.0	1920.379	1068.546	-591.987	832.054	842.147	1913.239	2509.724	3836.495	8308.235	1.149	2.329	10.133

```
# Physical-plausibility check – does each channel sit inside its expected Cessna 172 range?
plausibility = []
for ch in channel_names:
    col = f'{ch}__mean'
    if col not in flight_features_df.columns:
        continue
    vals = flight_features_df[col].dropna()
    if len(vals) == 0:
        continue
    lo, hi = PHYSICAL_RANGES.get(ch, (None, None))
    below = (vals < lo).sum() if lo is not None else 0
    above = (vals > hi).sum() if hi is not None else 0
    plausibility.append({
        'channel':      ch,
        'observed_min': round(vals.min(), 2),
        'observed_max': round(vals.max(), 2),
        'expected_lo':  lo,
        'expected_hi':  hi,
        'below_lo':     below,
        'above_hi':     above,
        'oor_pct':      round((below + above) / len(vals) * 100, 2), # out of range
    })
plausibility_df = pd.DataFrame(plausibility)
plausibility_df = plausibility_df.sort_values('oor_pct', ascending=False)
print('Physical-plausibility check (using each flight\'s mean):')
plausibility_df
```

Physical-plausibility check (using each flight's mean):

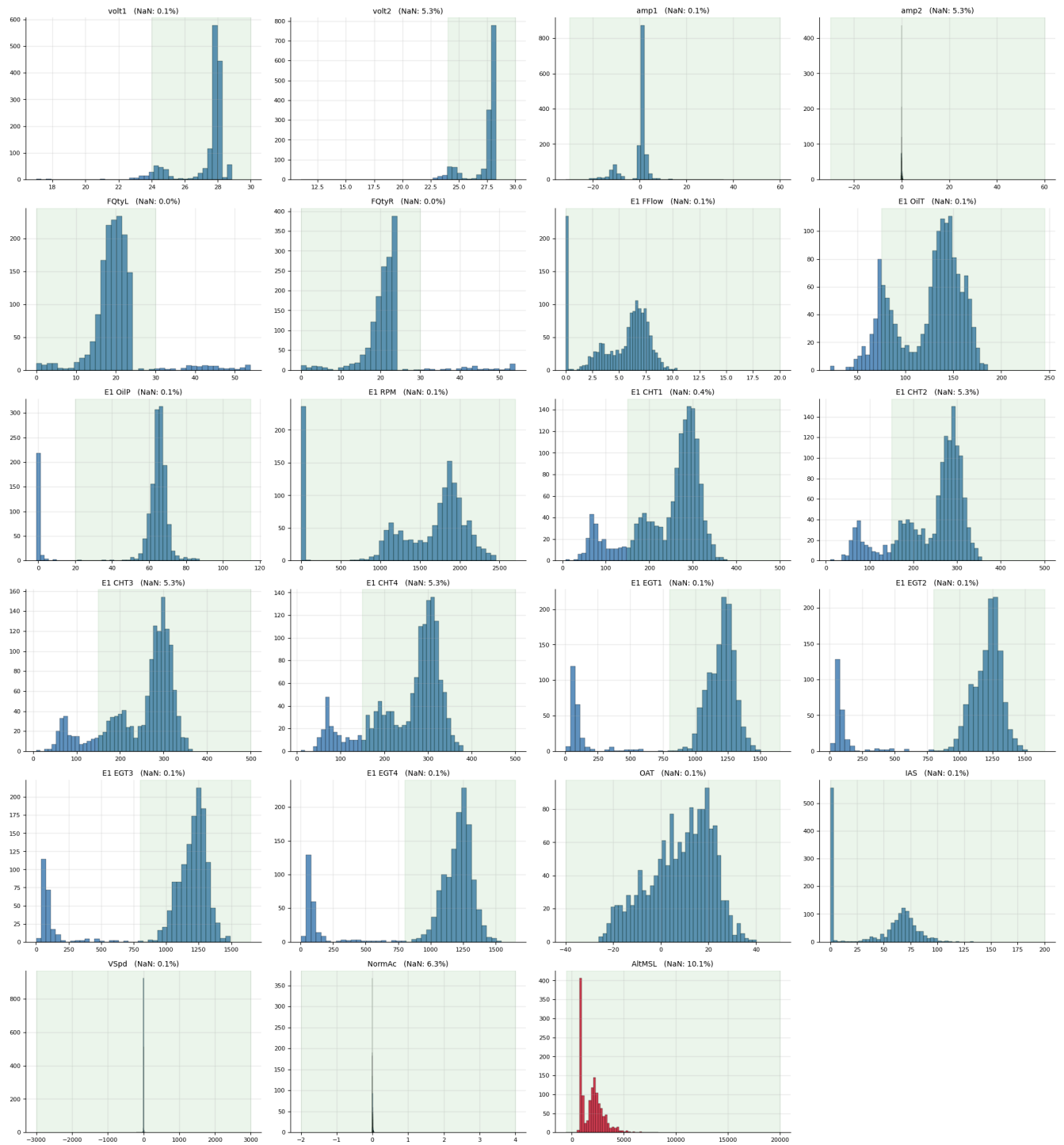
	channel	observed_min	observed_max	expected_lo	expected_hi	below_lo	above_hi	oor_pct
14	E1 EGT1	2.45	1501.88	800	1650	242	0	16.14
17	E1 EGT4	3.37	1545.34	800	1650	242	0	16.14
15	E1 EGT2	4.33	1520.27	800	1650	241	0	16.08
16	E1 EGT3	2.24	1492.24	800	1650	239	0	15.94
8	E1 OilP	-0.96	87.47	20	115	235	0	15.68
10	E1 CHT1	7.40	379.71	150	500	211	0	14.12
12	E1 CHT3	7.81	366.12	150	500	196	0	13.80
11	E1 CHT2	10.05	357.12	150	500	196	0	13.80
13	E1 CHT4	9.25	380.33	150	500	192	0	13.52
7	E1 OilT	21.70	185.97	75	245	194	0	12.95
4	FQtyL	0.00	54.00	0	30	0	69	4.60
5	FQtyR	0.00	54.00	0	30	0	69	4.60
0	volt1	17.02	28.87	24	30	55	0	3.67
1	volt2	11.02	28.30	24	30	52	0	3.66
2	amp1	-31.78	35.96	-30	60	1	0	0.07
22	AltMSL	-591.99	8308.23	-500	20000	1	0	0.07
3	amp2	-0.30	0.64	-30	60	0	0	0.00
6	E1 FFlow	0.00	10.44	0	20	0	0	0.00
9	E1 RPM	0.00	2456.16	0	2700	0	0	0.00
18	OAT	-26.00	40.15	-40	50	0	0	0.00
19	IAS	0.00	133.87	0	200	0	0	0.00
20	VSpd	-208.48	70.00	-3000	3000	0	0	0.00
21	NormAc	-0.01	0.05	-2	4	0	0	0.00

```
# Per-channel histogram grid – coloured by whether channel is high-missing
fig, axes = plt.subplots(6, 4, figsize=(20, 22))
axes = axes.flatten()

for i, ch in enumerate(channel_names):
    col = f'{ch}__mean'
    if col not in flight_features_df.columns:
        axes[i].set_visible(False); continue
    data = flight_features_df[col].dropna()
    nan_pct = flight_features_df[col].isna().mean() * 100
    colour = 'crimson' if nan_pct > 10 else 'steelblue'
    axes[i].hist(data, bins=40, color=colour, edgecolor='black', linewidth=0.3, alpha=0.85)
    # Add expected-range shading
    lo, hi = PHYSICAL_RANGES.get(ch, (None, None))
    if lo is not None and hi is not None:
        axes[i].axvspan(lo, hi, color='green', alpha=0.08, label='expected range')
    axes[i].set_title(f'{ch} (NaN: {nan_pct:.1f}%)', fontsize=10)
    axes[i].tick_params(labelsize=8)

for j in range(len(channel_names), len(axes)):
    axes[j].set_visible(False)
plt.suptitle('Per-channel distribution of flight-mean values (red title = high NaN)',
            fontsize=13, y=1.001)
plt.tight_layout(); plt.show()
```

Per-channel distribution of flight-mean values (red title = high NaN)



```
# Outlier rate per channel - IQR rule on flight means
outlier_rows = []
for ch in channel_names:
    col = f'{ch}_mean'
    if col not in flight_features_df.columns: continue
    s = flight_features_df[col].dropna()
```



```

q1, q3 = np.percentile(s, [25, 75])
iqr = q3 - q1
lo, hi = q1 - 1.5*iqr, q3 + 1.5*iqr
n_out = ((s < lo) | (s > hi)).sum()
outlier_rows.append({'channel': ch, 'IQR_outlier_count': n_out,
                    'IQR_outlier_pct': round(n_out/len(s)*100, 2)})
out_df = pd.DataFrame(outlier_rows).sort_values('IQR_outlier_pct', ascending=False)
print('Flight-level outlier rates (IQR rule on per-flight means):')
out_df

```

Flight-level outlier rates (IQR rule on per-flight means):

	channel	IQR_outlier_count	IQR_outlier_pct
2	amp1	316	21.08
3	amp2	291	20.49
0	volt1	301	20.08
8	E1 OilP	263	17.55
1	volt2	239	16.83
14	E1 EGT1	241	16.08
17	E1 EGT4	241	16.08
16	E1 EGT3	239	15.94
15	E1 EGT2	239	15.94
5	FQtyR	149	9.93
20	VSpd	144	9.61
4	FQtyL	129	8.60
21	NormAc	113	8.04
22	AltMSL	18	1.34
12	E1 CHT3	11	0.77
10	E1 CHT1	8	0.54
11	E1 CHT2	5	0.35
13	E1 CHT4	2	0.14
6	E1 FFlow	0	0.00
7	E1 OilT	0	0.00
9	E1 RPM	0	0.00
18	OAT	0	0.00
19	IAS	0	0.00

7. Missing-Value Analysis — Pattern, Co-occurrence, and Class-Conditioning

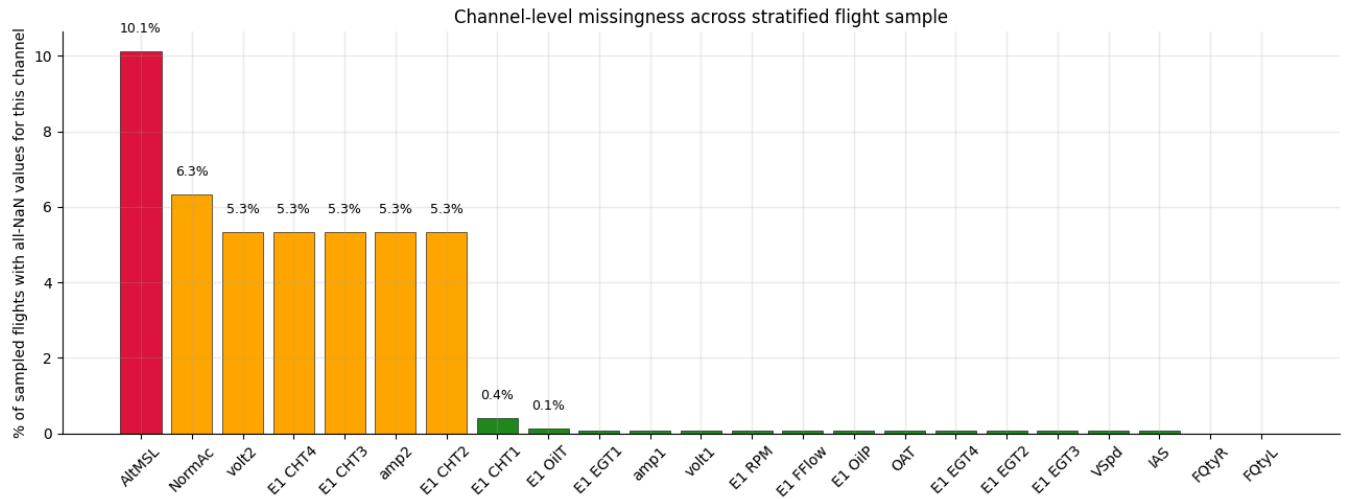
```

# (a) Per-flight NaN fraction per channel, using the flight aggregates
# A flight is "missing" channel X if ALL timesteps of that channel are NaN.
flight_level_missing = pd.DataFrame(index=flight_features_df.index)
for ch in channel_names:
    mean_col = f'{ch}_mean'
    if mean_col in flight_features_df.columns:
        flight_level_missing[ch] = flight_features_df[mean_col].isna()

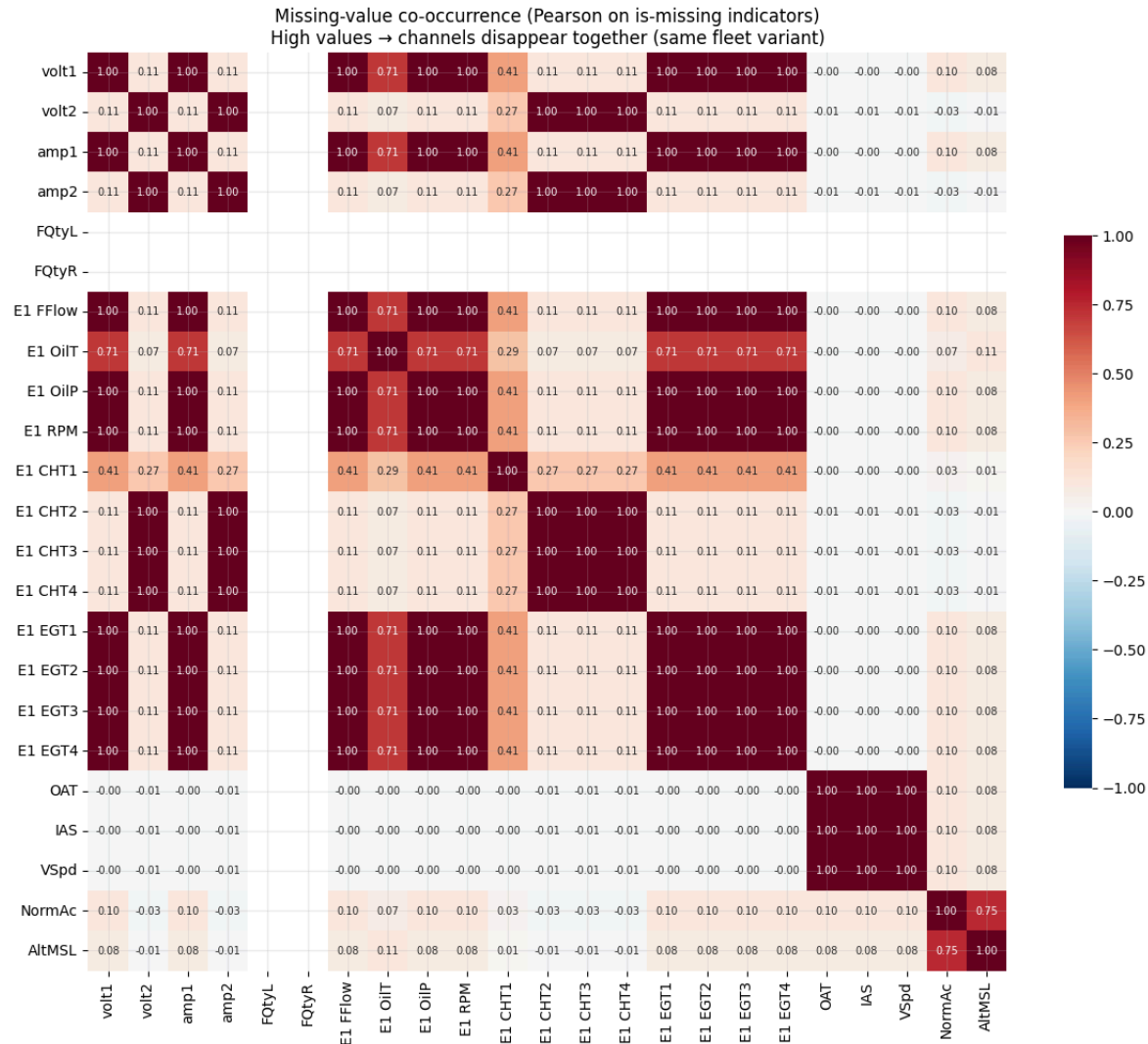
miss_pct = flight_level_missing.mean() * 100
miss_pct = miss_pct.sort_values(ascending=False)

fig, ax = plt.subplots(figsize=(13, 5))
colors = ['crimson' if v > 10 else 'orange' if v > 1 else 'forestgreen' for v in miss_pct.values]
ax.bar(miss_pct.index, miss_pct.values, color=colors, edgecolor='black', linewidth=0.4)
ax.set_ylabel('% of sampled flights with all-NaN values for this channel')
ax.set_title('Channel-level missingness across stratified flight sample')
ax.tick_params(axis='x', rotation=45)
for i, v in enumerate(miss_pct.values):
    if v > 0.1:
        ax.text(i, v + 0.5, f'{v:.1f}%', ha='center', fontsize=9)
plt.tight_layout(); plt.show()

```



```
# (b) Co-occurrence of missingness – correlation of missing indicators
miss_corr = flight_level_missing.corr()
fig, ax = plt.subplots(figsize=(12, 10))
sns.heatmap(miss_corr, cmap='RdBu_r', center=0, vmin=-1, vmax=1,
            square=True, annot=True, fmt='.2f', annot_kws={'size':7}, cbar_kws={'shrink':0.6}, ax=ax)
ax.set_title('Missing-value co-occurrence (Pearson on is-missing indicators)\n'
            'High values → channels disappear together (same fleet variant)')
plt.tight_layout(); plt.show()
```



```
# (c) missing rate per channel, broken down by class
miss_by_class = (flight_level_missing
                .join(flight_features_df['before_after'])
                .groupby('before_after'))
```

```

        .mean() * 100)
print('% of flights missing each channel, by before_after class:')
miss_by_class.round(2)

```

% of flights missing each channel, by before_after class:

	volt1	volt2	amp1	amp2	FQtyL	FQtyR	E1 FFlow	E1 OilT	E1 OilP	E1 RPM	...	E1 CHT4	E1 EGT1	E1 EGT2	E1 EGT3	E1 EGT4	OAT	IAS	VSpd	NormAc	AltMSL
before_after																					
after	0.0	5.0	0.0	5.0	0.0	0.0	0.0	0.0	0.0	0.0	...	5.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	4.4	7.0
before	0.0	4.6	0.0	4.6	0.0	0.0	0.0	0.2	0.0	0.0	...	4.6	0.0	0.0	0.0	0.0	0.0	0.0	0.0	4.8	8.2
same	0.2	6.4	0.2	6.4	0.0	0.0	0.2	0.2	0.2	0.2	...	6.4	0.2	0.2	0.2	0.2	0.2	0.2	0.2	9.8	15.2

8. Correlation Analysis — Global, Class-Conditional, and Redundancy Groups

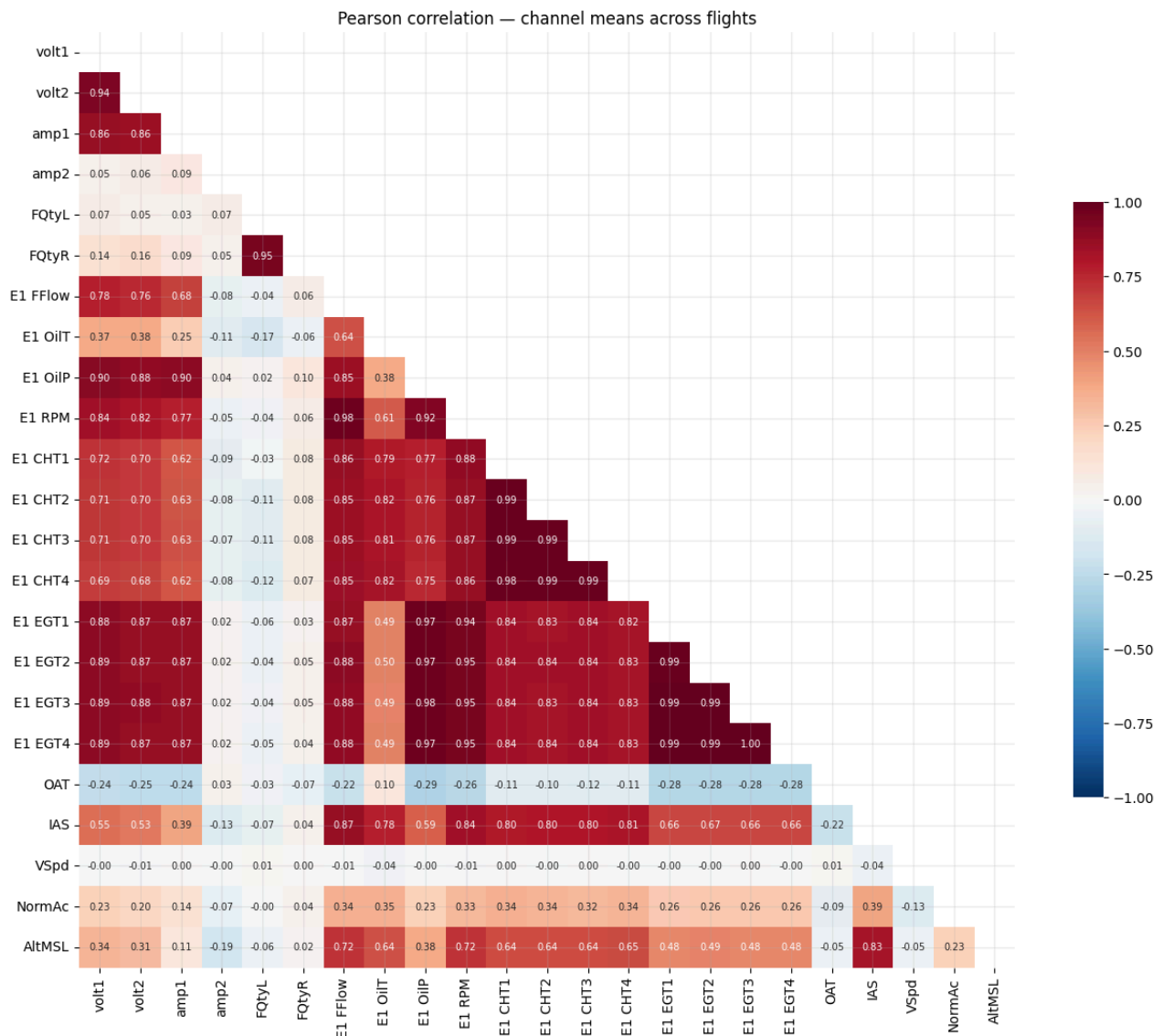
The teammate's correlation matrix used raw timesteps. We redo it on **flight-level means** (one row per flight) and **class-condition** it to check whether sensor relationships shift between before/after maintenance.

```

# Global correlation on flight-level means
corr_global = flight_features_df[mean_cols].corr()
corr_global.index = [c.replace('__mean__', '') for c in corr_global.index]
corr_global.columns = [c.replace('__mean__', '') for c in corr_global.columns]

fig, ax = plt.subplots(figsize=(13, 11))
mask = np.triu(np.ones_like(corr_global, dtype=bool))
sns.heatmap(corr_global, mask=mask, cmap='RdBu_r', center=0, vmin=-1, vmax=1,
            square=True, annot=True, fmt='.2f', annot_kws={'size':7},
            cbar_kws={'shrink':0.6}, ax=ax)
ax.set_title('Pearson correlation - channel means across flights')
plt.tight_layout(); plt.show()

```



```
# Find redundancy groups: pairs with |r| > 0.9
```

```

high_corr_pairs = []
for i, j in combinations(corr_global.columns, 2):
    r = corr_global.loc[i, j]
    if abs(r) > 0.9:
        high_corr_pairs.append({'channel_a': i, 'channel_b': j, 'r': round(r, 3)})
high_corr_df = pd.DataFrame(high_corr_pairs).sort_values('r', key=abs, ascending=False)
print(f'Highly-correlated channel pairs (|r| > 0.9): {len(high_corr_df)}')
high_corr_df

```

Highly-correlated channel pairs (|r| > 0.9): 25

	channel_a	channel_b	r
24	E1 EGT3	E1 EGT4	0.996
22	E1 EGT2	E1 EGT3	0.994
20	E1 EGT1	E1 EGT3	0.994
23	E1 EGT2	E1 EGT4	0.994
21	E1 EGT1	E1 EGT4	0.992
17	E1 CHT2	E1 CHT4	0.990
19	E1 EGT1	E1 EGT2	0.990
16	E1 CHT2	E1 CHT3	0.989
18	E1 CHT3	E1 CHT4	0.989
14	E1 CHT1	E1 CHT3	0.987
13	E1 CHT1	E1 CHT2	0.986
15	E1 CHT1	E1 CHT4	0.983
3	E1 FFlow	E1 RPM	0.976
7	E1 OilP	E1 EGT3	0.976
8	E1 OilP	E1 EGT4	0.974
6	E1 OilP	E1 EGT2	0.972
5	E1 OilP	E1 EGT1	0.972
2	FQtyL	FQtyR	0.952
10	E1 RPM	E1 EGT2	0.950
11	E1 RPM	E1 EGT3	0.950
12	E1 RPM	E1 EGT4	0.949
9	E1 RPM	E1 EGT1	0.945
0	volt1	volt2	0.937
4	E1 OilP	E1 RPM	0.920
1	volt1	E1 OilP	0.901

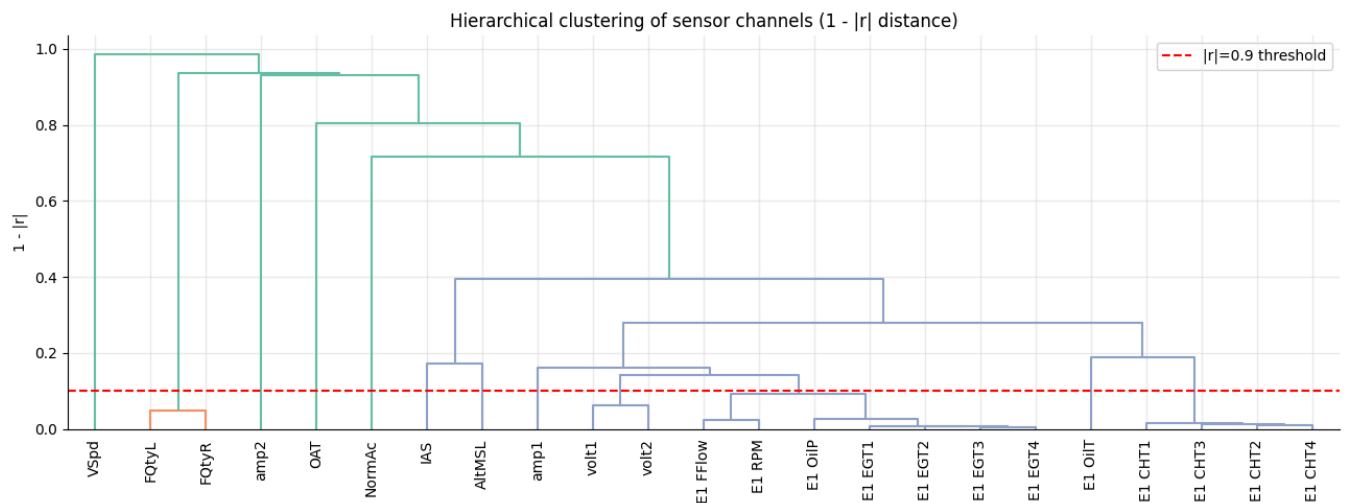
```

# Hierarchical clustering of channels - visualises redundancy groups
from scipy.cluster.hierarchy import linkage, dendrogram

corr_clean = corr_global.dropna(how='all').dropna(axis=1, how='all').fillna(0)
dist = 1 - corr_clean.abs()
Z = linkage(dist.values[np.triu_indices_from(dist.values, k=1)], method='average')

fig, ax = plt.subplots(figsize=(13, 5))
dendrogram(Z, labels=corr_clean.index.tolist(), leaf_rotation=90, ax=ax)
ax.set_title('Hierarchical clustering of sensor channels (1 - |r| distance)')
ax.set_ylabel('1 - |r|')
ax.axhline(0.1, color='red', ls='--', label='|r|=0.9 threshold')
ax.legend()
plt.tight_layout(); plt.show()

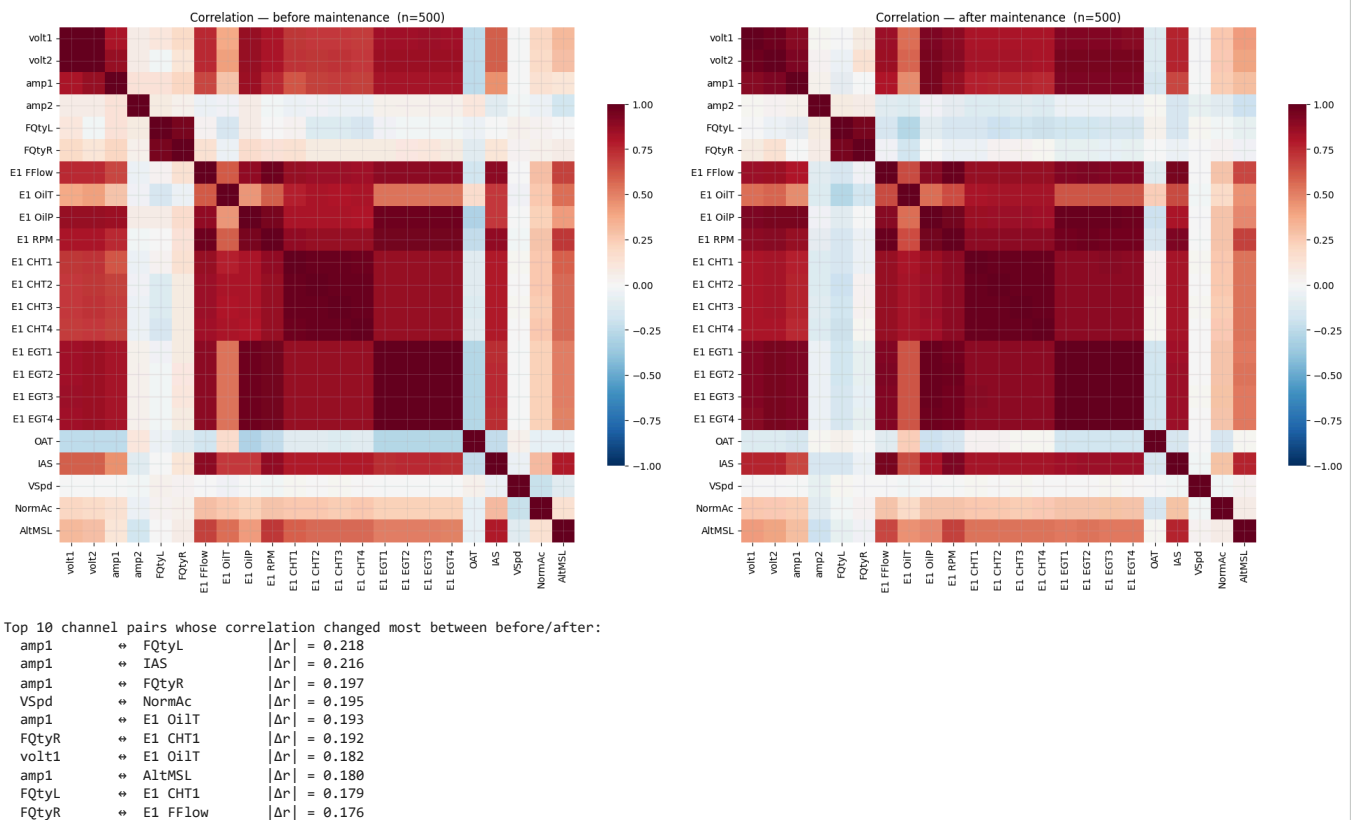
```



```
# Class-conditional correlation – do channel relationships SHIFT between before/after?
classes_for_cc = [c for c in ['before', 'after'] if c in flight_features_df['before_after'].unique()]
fig, axes = plt.subplots(1, 2, figsize=(20, 9))

for ax, cls in zip(axes, classes_for_cc):
    sub = flight_features_df[flight_features_df['before_after'] == cls]
    cc = sub[mean_cols].corr()
    cc.index = [c.replace('__mean', '') for c in cc.index]
    cc.columns = [c.replace('__mean', '') for c in cc.columns]
    sns.heatmap(cc, cmap='RdBu_r', center=0, vmin=-1, vmax=1,
                square=True, cbar_kws={'shrink':0.6}, ax=ax)
    ax.set_title(f'Correlation – {cls} maintenance (n={len(sub):,})')
plt.tight_layout(); plt.show()

# Δ correlation – which pairs change the most?
if len(classes_for_cc) == 2:
    c_b = flight_features_df[flight_features_df['before_after']=='before'][mean_cols].corr()
    c_a = flight_features_df[flight_features_df['before_after']=='after'][mean_cols].corr()
    delta = (c_a - c_b).abs()
    delta.index = [c.replace('__mean', '') for c in delta.index]
    delta.columns = [c.replace('__mean', '') for c in delta.columns]
    # Get top 10 pairs that changed the most
    pairs = []
    for i, j in combinations(delta.columns, 2):
        pairs.append((i, j, delta.loc[i, j]))
    top_shifts = sorted(pairs, key=lambda x: -x[2][:10])
    print('\nTop 10 channel pairs whose correlation changed most between before/after:')
    for a, b, d in top_shifts:
        print(f' {a:10s} ↔ {b:10s} |Δr| = {d:.3f}')
```



9. Class-Conditional Sensor Analysis — KS, Mann-Whitney, Cohen's d

```
# Build the test table
def cohens_d(a, b):
    a = np.asarray(a, dtype=float); b = np.asarray(b, dtype=float)
    a = a[~np.isnan(a)]; b = b[~np.isnan(b)]
    if len(a) < 2 or len(b) < 2: return np.nan
    n1, n2 = len(a), len(b)
    s_pool = np.sqrt(((n1-1)*a.var(ddof=1) + (n2-1)*b.var(ddof=1)) / (n1+n2-2))
    return (a.mean() - b.mean()) / s_pool if s_pool > 0 else np.nan

# Test each {channel}__{stat} aggregate
test_results = []
agg_stats_to_test = ['mean', 'std', 'max', 'min', 'range', 'slope']
before_mask = flight_features_df['before_after'] == 'before'
```

```

after_mask = flight_features_df['before_after'] == 'after'

for ch in channel_names:
    for stat in agg_stats_to_test:
        col = f'{ch}_{stat}'
        if col not in flight_features_df.columns: continue
        a = flight_features_df.loc[before_mask, col].dropna()
        b = flight_features_df.loc[after_mask, col].dropna()
        if len(a) < 30 or len(b) < 30: continue
        try:
            ks_stat, ks_p = ks_2samp(a, b)
            mw_stat, mw_p = mannwhitneyu(a, b, alternative='two-sided')
        except Exception:
            continue
        test_results.append({
            'feature': col,
            'channel': ch,
            'stat': stat,
            'n_before': len(a),
            'n_after': len(b),
            'mean_before': a.mean(),
            'mean_after': b.mean(),
            'cohens_d': cohens_d(a, b),
            'ks_p': ks_p,
            'mw_p': mw_p,
        })

tests_df = pd.DataFrame(test_results)
# Bonferroni correction over the family of tests
tests_df['ks_p_bonf'] = (tests_df['ks_p'] * len(tests_df)).clip(upper=1)
tests_df['mw_p_bonf'] = (tests_df['mw_p'] * len(tests_df)).clip(upper=1)
tests_df = tests_df.assign(abs_d=tests_df['cohens_d'].abs()).sort_values('abs_d', ascending=False)
print(f'Features tested: {len(tests_df)}')
print(f'Significant after Bonferroni (KS): {(tests_df["ks_p_bonf"]<0.05).sum()}')
print(f'\nTop 20 features by |Cohen's d|:')
tests_df.head(20)[['feature', 'mean_before', 'mean_after', 'cohens_d', 'ks_p_bonf', 'mw_p_bonf']].round(4)

```

Features tested: 138
Significant after Bonferroni (KS): 2

Top 20 features by |Cohen's d|:

	feature	mean_before	mean_after	cohens_d	ks_p_bonf	mw_p_bonf
83	E1 CHT4_slope	0.0601	0.0245	0.2157	0.8184	1.0000
65	E1 CHT1_slope	0.0576	0.0237	0.2100	1.0000	1.0000
71	E1 CHT2_slope	0.0556	0.0233	0.2090	1.0000	1.0000
15	amp1_min	-12.1156	-13.1828	0.1952	1.0000	0.9447
47	E1 OilT_slope	0.0112	0.0083	0.1646	1.0000	1.0000
61	E1 CHT1_std	36.3316	33.6145	0.1532	1.0000	1.0000
111	OAT_min	4.5528	2.6210	0.1448	1.0000	1.0000
77	E1 CHT3_slope	0.0794	0.0243	0.1423	1.0000	1.0000
73	E1 CHT3_std	35.5320	33.1043	0.1391	1.0000	1.0000
63	E1 CHT1_min	114.6269	107.9230	0.1379	1.0000	1.0000
108	OAT_mean	8.0096	6.2207	0.1360	1.0000	1.0000
45	E1 OilT_min	98.0629	93.8474	0.1329	0.8921	1.0000
120	VSpd_mean	-1.2141	-0.0307	-0.1315	1.0000	1.0000
79	E1 CHT4_std	36.7044	34.3741	0.1301	1.0000	1.0000
129	NormAc_min	-0.3500	-0.3734	0.1237	1.0000	1.0000
124	VSpd_range	2758.1807	2979.5966	-0.1219	1.0000	1.0000
28	FQtyL_range	7.9659	8.6098	-0.1216	1.0000	1.0000
67	E1 CHT2_std	36.1448	34.0276	0.1214	1.0000	1.0000
122	VSpd_max	1342.0045	1437.8660	-0.1178	1.0000	1.0000
25	FQtyL_std	1.8872	2.0451	-0.1166	1.0000	1.0000

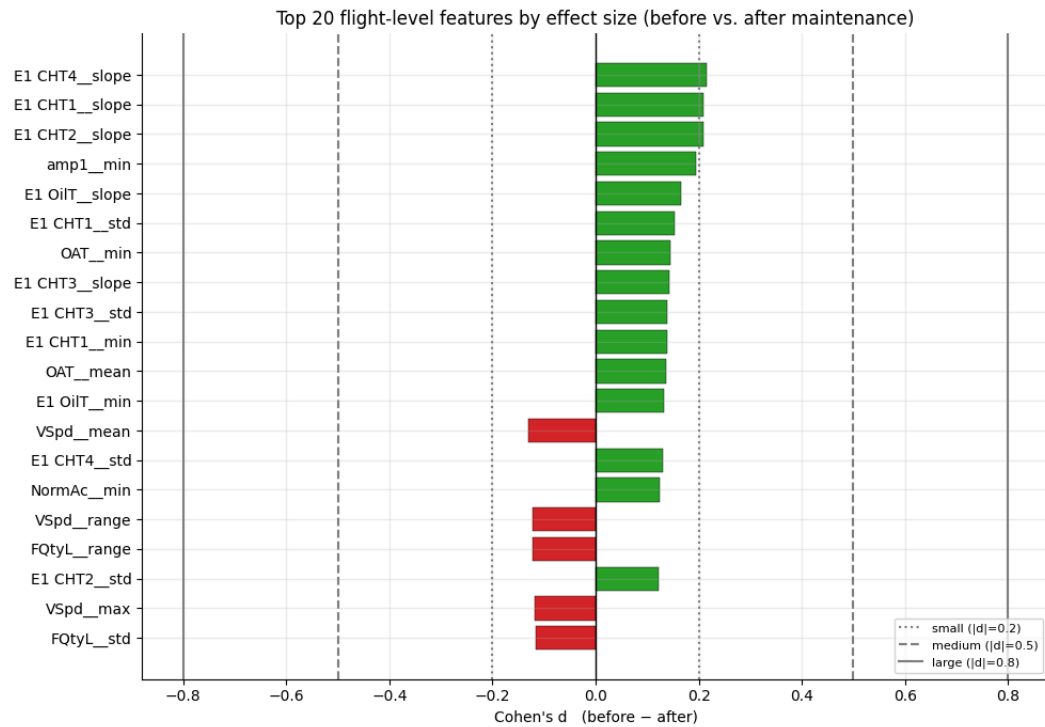
```

# Visualise the top-discriminating features as a forest plot of effect sizes
top_n = 20
top_features = tests_df.head(top_n).copy()
top_features = top_features.iloc[::-1] # so largest sits on top after barh

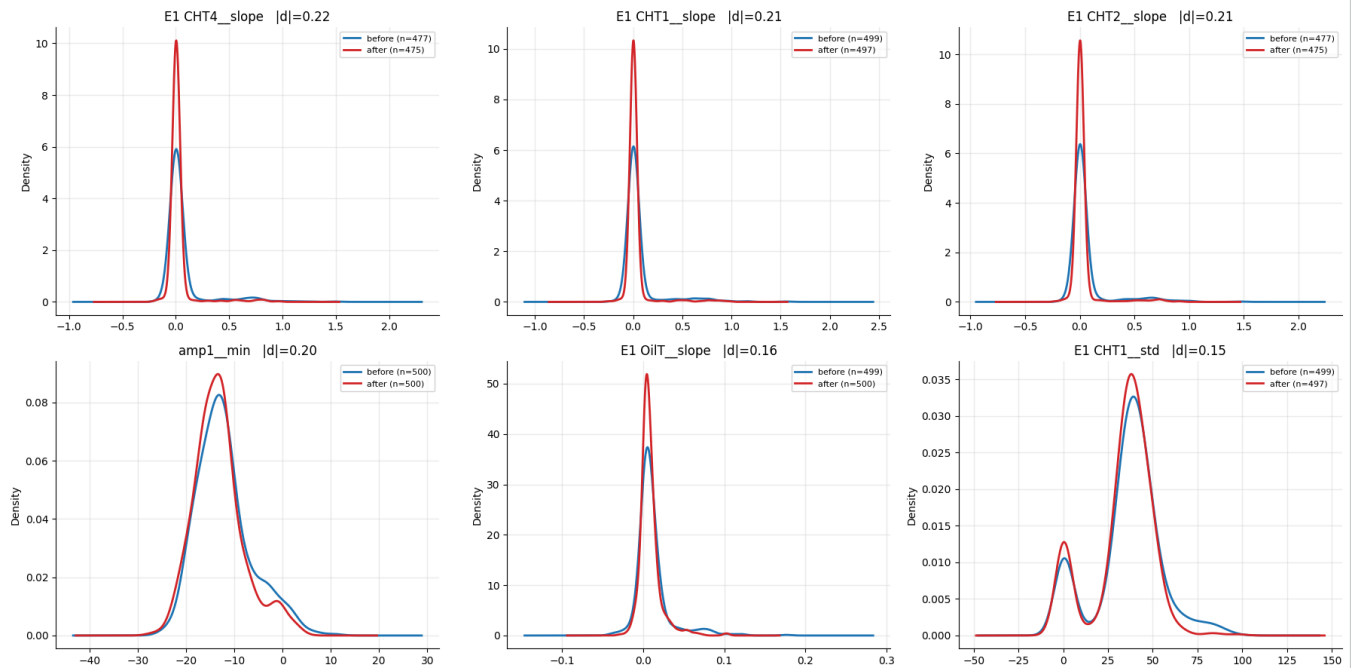
fig, ax = plt.subplots(figsize=(10, max(6, top_n*0.35)))
colors = ['#d62728' if d < 0 else '#2ca02c' for d in top_features['cohens_d']]
ax.barh(top_features['feature'], top_features['cohens_d'], color=colors,
        edgecolor='black', linewidth=0.3)
ax.axvline(0, color='black', linewidth=1)
ax.axvline(0.2, color='gray', ls=':', label='small (|d|=0.2)')
ax.axvline(-0.2, color='gray', ls=':')
ax.axvline(0.5, color='gray', ls='--', label='medium (|d|=0.5)')
ax.axvline(-0.5, color='gray', ls='--')
ax.axvline(0.8, color='gray', ls='-', label='large (|d|=0.8)')
ax.axvline(-0.8, color='gray', ls='-')
ax.set_xlabel("Cohen's d (before - after)")
ax.set_title(f'Top {top_n} flight-level features by effect size (before vs. after maintenance)')

```

```
ax.legend(loc='lower right', fontsize=8)
plt.tight_layout(); plt.show()
```



```
# KDE comparison for the top 6 most-discriminating features
fig, axes = plt.subplots(2, 3, figsize=(18, 9))
for ax, (_, row) in zip(axes.flatten(), tests_df.head(6).iterrows()):
    col = row['feature']
    for cls, color in [('before', '#1f77b4'), ('after', '#d62728')]:
        sub = flight_features_df.loc[flight_features_df['before_after']==cls, col].dropna()
        if len(sub) > 0:
            sub.plot.kde(ax=ax, label=f'{cls} (n={len(sub)})', color=color, linewidth=2)
    ax.set_title(f'{col} |d|={abs(row["cohens_d"]):.2f}')
    ax.legend(fontsize=8)
plt.tight_layout(); plt.show()
```



10. Time-Series-Specific Properties

```
# Inspect raw timestep structure on a single flight
inspect_idx = sampled_indices[0]
arr0 = load_flight_array(inspect_idx)
arr0_df = pd.DataFrame(arr0[:, :len(channel_names)], columns=channel_names)
print(f'Flight {inspect_idx}: shape = {arr0.shape}')
print(f'Class: {flight_header_df.loc[inspect_idx, "before_after"]} | Label: {flight_header_df.loc[inspect_idx, "label"]}')
arr0_df.head(3)
```

Flight 58: shape = (5337, 23)
Class: after | Label: intake gasket leak/damage

	volt1	volt2	amp1	amp2	FQtyL	FQtyR	E1 FFlow	E1 OilT	E1 OilP	E1 RPM	...	E1 CHT4	E1 EGT1	E1 EGT2	E1 EGT3	E1 EGT4	OAT	IAS	VSpd	NormAc	AltMSL
0	28.0	28.0	3.6	0.3	20.67	21.95	3.04	61.39	64.71	1187.4	...	63.54	936.45	979.44	950.61	961.72	12.8	0.0	-1.79	0.01	837.2
1	28.0	28.0	3.6	0.3	20.67	21.95	3.09	61.40	65.04	1206.9	...	63.74	944.32	984.85	953.75	967.39	12.8	0.0	7.62	0.02	837.0
2	28.0	28.0	3.6	0.3	20.62	21.95	3.06	61.43	64.80	1200.9	...	64.30	956.35	995.98	958.93	977.21	12.5	0.0	2.16	0.02	NaN

3 rows × 23 columns

```
# Within-flight variability
cv_records = []
for ch in channel_names:
    mean_col = f'{ch}__mean'
    std_col = f'{ch}__std'
    if mean_col in flight_features_df.columns and std_col in flight_features_df.columns:
        cv = (flight_features_df[std_col] /
              flight_features_df[mean_col].abs().replace(0, np.nan)).dropna()
        cv_records.append({'channel': ch, 'cv_median': cv.median(), 'cv_p95': cv.quantile(0.95)})
cv_df = pd.DataFrame(cv_records).sort_values('cv_median', ascending=False)
print('Within-flight coefficient of variation per channel (median across flights):')
cv_df
```

Within-flight coefficient of variation per channel (median across flights):

	channel	cv_median	cv_p95
20	VSpd	262.950560	3376.771940
21	NormAc	19.120975	210.266523
3	amp2	1.938701	29.154938
2	amp1	1.551094	10.405749
19	IAS	0.628481	4.246637
6	E1 FFlow	0.616875	0.991292
22	AltMSL	0.454599	0.657851
9	E1 RPM	0.332116	0.431041
18	OAT	0.155064	1.766570
11	E1 CHT2	0.139757	0.393123
10	E1 CHT1	0.136705	0.385915
13	E1 CHT4	0.134091	0.400425
12	E1 CHT3	0.131279	0.393677
15	E1 EGT2	0.121792	0.184296
14	E1 EGT1	0.118443	0.184817
16	E1 EGT3	0.117869	0.186556
17	E1 EGT4	0.116945	0.179178
8	E1 OilP	0.096130	0.164201
7	E1 OilT	0.095439	0.234098
4	FQtyL	0.090538	0.210091
5	FQtyR	0.040511	0.175939
1	volt2	0.014254	0.040208
0	volt1	0.013928	0.039799

```
# Autocorrelation function (ACF) for key channels on a few example flights
from statsmodels.graphics.tsaplots import plot_acf

key_channels = ['E1 RPM', 'E1 OilT', 'E1 EGT1', 'IAS', 'AltMSL', 'NormAc']
n_examples = 3
example_idx = rng_local.choice(sampled_indices, size=n_examples, replace=False)

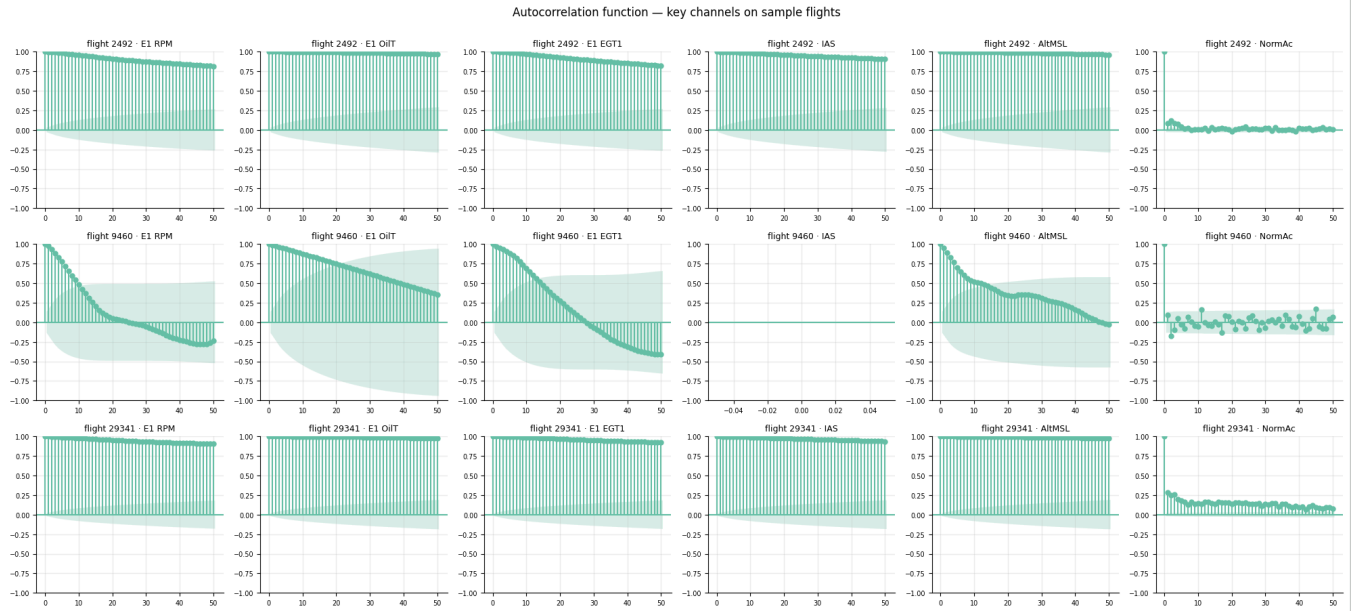
fig, axes = plt.subplots(n_examples, len(key_channels), figsize=(20, 3*n_examples))
for r, idx in enumerate(example_idx):
    arr = load_flight_array(idx)
    arr_df = pd.DataFrame(arr[:, :len(channel_names)], columns=channel_names)
    for c, ch in enumerate(key_channels):
        if ch not in arr_df.columns or arr_df[ch].isna().all():
            axes[r, c].set_visible(False); continue
        series = arr_df[ch].dropna()
        if len(series) < 50:
            axes[r, c].set_visible(False); continue
        lag_max = min(50, len(series) // 4)
```



```

plot_acf(series, lags=lag_max, ax=axes[r, c], title='')
axes[r, c].set_title(f'flight {idx} · {ch}', fontsize=9)
axes[r, c].tick_params(labelsize=7)
plt.suptitle('Autocorrelation function — key channels on sample flights', fontsize=12, y=1.001)
plt.tight_layout(); plt.show()

```



```

# Flight-phase identification — use RPM + IAS + AltMSL to segment
def identify_phases(arr_df):
    """Heuristic phase labels: ground / taxi / takeoff / climb / cruise / descent / landing."""
    rpm = arr_df.get('E1 RPM', pd.Series(np.nan, index=arr_df.index))
    ias = arr_df.get('IAS', pd.Series(np.nan, index=arr_df.index))
    vspd = arr_df.get('VSpd', pd.Series(np.nan, index=arr_df.index))
    phases = np.full(len(arr_df), 'unknown', dtype=object)
    phases[(rpm < 1000) & (ias < 5)] = 'ground'
    phases[(rpm >= 1000) & (rpm < 1800) & (ias < 40)] = 'taxi'
    phases[(rpm >= 1800) & (ias >= 40) & (ias < 70)] = 'takeoff'
    phases[(ias >= 70) & (vspd > 300)] = 'climb'
    phases[(ias >= 70) & (vspd.abs() <= 300)] = 'cruise'
    phases[(ias >= 60) & (vspd < -300)] = 'descent'
    phases[(rpm < 1800) & (ias >= 40) & (ias < 70) & (vspd < 0)] = 'landing'
    return phases

# Apply to one example flight & visualise
arr1 = load_flight_array(sampled_indices[0])
arr1_df = pd.DataFrame(arr1[:, :len(channel_names)], columns=channel_names)
phases = identify_phases(arr1_df)

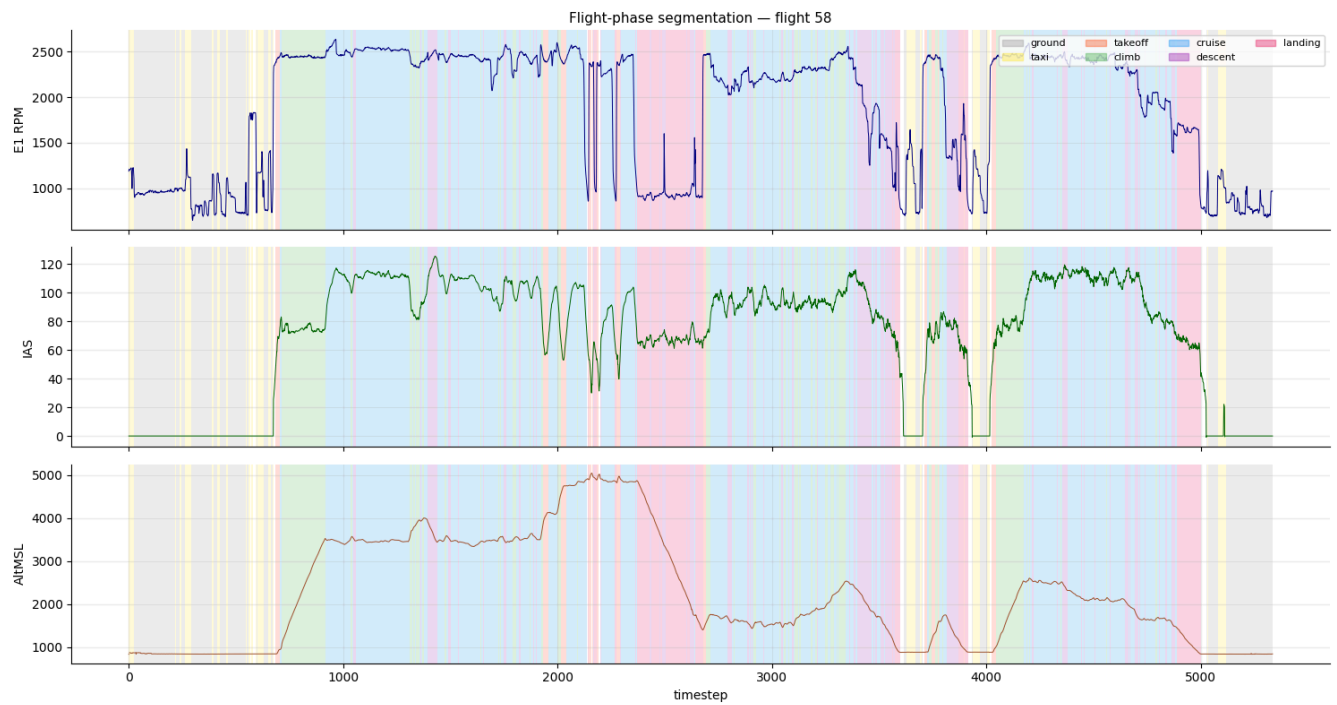
phase_colors = {
    'ground': '#9e9e9e', 'taxi': '#ffeb3b', 'takeoff': '#ff5722',
    'climb': '#4caf50', 'cruise': '#2196f3', 'descent': '#9c27b0',
    'landing': '#e91e63', 'unknown': '#000000'
}

fig, axes = plt.subplots(3, 1, figsize=(15, 8), sharex=True)
for ax, (ch, color) in zip(axes, [('E1 RPM', 'navy'), ('IAS', 'darkgreen'), ('AltMSL', 'sienna')]):
    if ch in arr1_df.columns:
        ax.plot(arr1_df[ch].values, linewidth=0.7, color=color)
        ax.set_ylabel(ch)
# Background colour by phase
for i in range(len(phases)-1):
    if phases[i] != 'unknown':
        for ax in axes:
            ax.axvspan(i, i+1, color=phase_colors[phases[i]], alpha=0.18, linewidth=0)
axes[-1].set_xlabel('timestep')

# Build legend manually
from matplotlib.patches import Patch
handles = [Patch(color=c, alpha=0.4, label=p) for p, c in phase_colors.items()]
if p in pd.unique(phases) and p != 'unknown':
    axes[0].legend(handles=handles, loc='upper right', fontsize=8, ncol=4)
axes[0].set_title(f'Flight-phase segmentation — flight {sampled_indices[0]}', fontsize=11)
plt.tight_layout(); plt.show()

print('\nPhase composition of this flight:')
print(pd.Series(phases).value_counts(normalize=True).round(3).rename('fraction'))

```



11. Dimensionality Reduction: PCA & UMAP

With 23 channels × 10 stats = ~230 features per flight (after dropping `last`), we want to know:

- Is most of the variance in a few components? (PCA scree)
- Do classes separate naturally in the reduced space? (PCA & UMAP scatter)
- Which raw channels drive the top components? (PCA loadings)

```
# Build the numeric feature matrix — keep only columns with <30% missing, impute the rest with median
feature_cols_all = [c for c in flight_features_df.columns
                    if any(c.endswith(f'_{s}') for s in ['mean', 'std', 'max', 'min', 'p05', 'p50', 'p95', 'range', 'slope'])]
```

```
X = flight_features_df[feature_cols_all].copy()
keep = X.columns[X.isna().mean() < 0.30]
X = X[keep].fillna(X[keep].median())
```

```
y_ba = flight_features_df['before_after'].values
y_label = flight_features_df['label'].values
```

```
# Scale before PCA
X_scaled = RobustScaler().fit_transform(X)
print(f'Feature matrix for PCA/UMAP: {X.shape} (n_samples × n_features)')
```

Feature matrix for PCA/UMAP: (1500, 207) (n_samples × n_features)

```
# PCA — scree plot + cumulative variance
pca_full = PCA(n_components=min(30, X.shape[1]))
pca_full.fit(X_scaled)
```

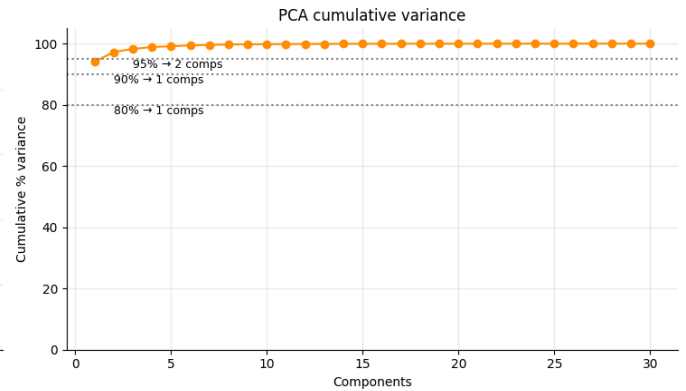
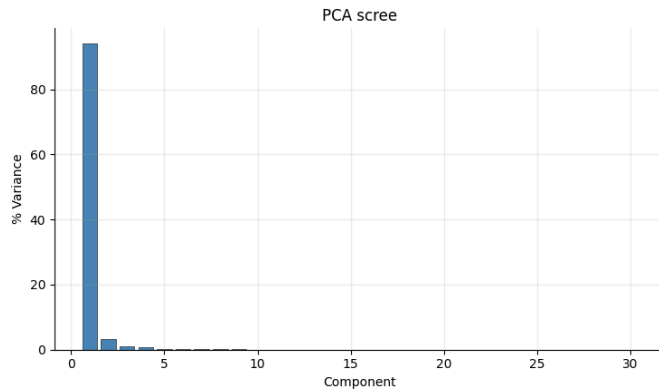
```
fig, axes = plt.subplots(1, 2, figsize=(15, 4.5))
axes[0].bar(range(1, len(pca_full.explained_variance_ratio_)+1),
            pca_full.explained_variance_ratio_ * 100,
            color='steelblue', edgecolor='black', linewidth=0.4)
axes[0].set_xlabel('Component'); axes[0].set_ylabel('% Variance')
axes[0].set_title('PCA scree')
```

```
cum = np.cumsum(pca_full.explained_variance_ratio_) * 100
axes[1].plot(range(1, len(cum)+1), cum, marker='o', color='darkorange')
```

```

for thresh in [80, 90, 95]:
    n = int(np.searchsorted(cum, thresh) + 1)
    axes[1].axhline(thresh, ls=':', color='gray')
    axes[1].annotate(f'{thresh}% → {n} comps', xy=(n, thresh), xytext=(n+1, thresh-3), fontsize=9)
axes[1].set_xlabel('Components')
axes[1].set_ylabel('Cumulative % variance')
axes[1].set_title('PCA cumulative variance')
axes[1].set_ylim(0, 105)
plt.tight_layout(); plt.show()

```



```

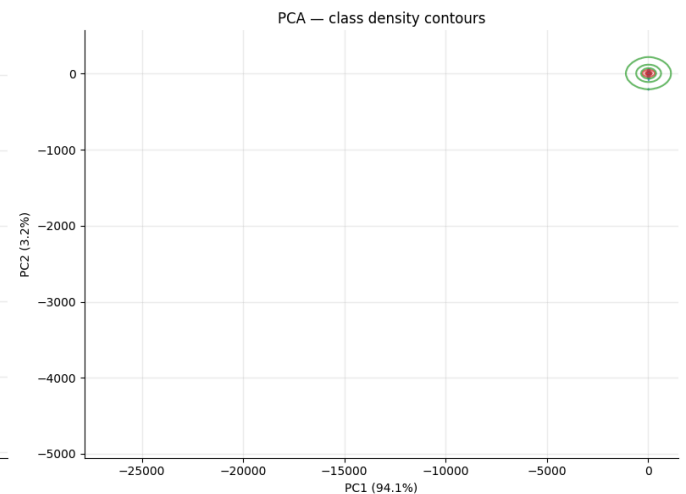
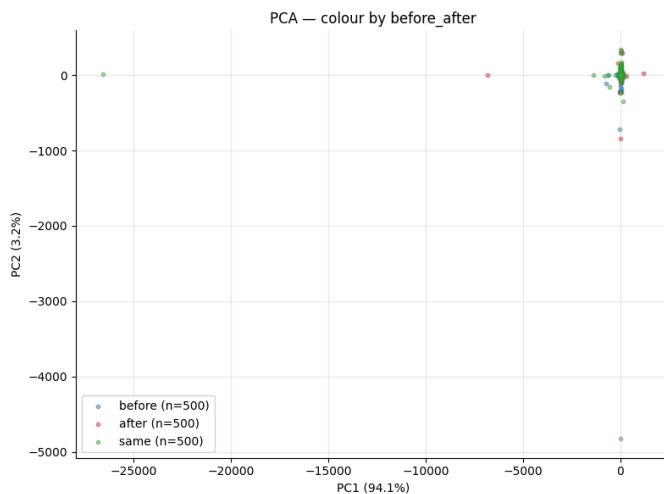
# PCA 2-D scatter coloured by before_after, with KDE overlay
pca2 = PCA(n_components=2)
X_pca = pca2.fit_transform(X_scaled)

fig, axes = plt.subplots(1, 2, figsize=(16, 6))
palette = {'before': '#1f77b4', 'after': '#d62728', 'same': '#2ca02c'}

# (a) Scatter coloured by class
for cls, color in palette.items():
    mask = y_ba == cls
    if mask.any():
        axes[0].scatter(X_pca[mask, 0], X_pca[mask, 1], s=10, alpha=0.45,
                        color=color, label=f'{cls} (n={mask.sum()})')
axes[0].set_xlabel(f'PC1 ({pca2.explained_variance_ratio_[0]*100:.1f}%)')
axes[0].set_ylabel(f'PC2 ({pca2.explained_variance_ratio_[1]*100:.1f}%)')
axes[0].set_title('PCA - colour by before_after')
axes[0].legend()

# (b) Density (KDE) per class
for cls, color in palette.items():
    mask = y_ba == cls
    if mask.sum() > 20:
        sns.kdeplot(x=X_pca[mask, 0], y=X_pca[mask, 1], levels=4, color=color, ax=axes[1], alpha=0.7)
axes[1].set_xlabel(f'PC1 ({pca2.explained_variance_ratio_[0]*100:.1f}%)')
axes[1].set_ylabel(f'PC2 ({pca2.explained_variance_ratio_[1]*100:.1f}%)')
axes[1].set_title('PCA - class density contours')
plt.tight_layout(); plt.show()

```



```

# PCA loadings: which features drive PC1 and PC2?
loadings = pd.DataFrame(pca2.components_.T, columns=['PC1', 'PC2'], index=X.columns)
loadings['abs_PC1'] = loadings['PC1'].abs()

```

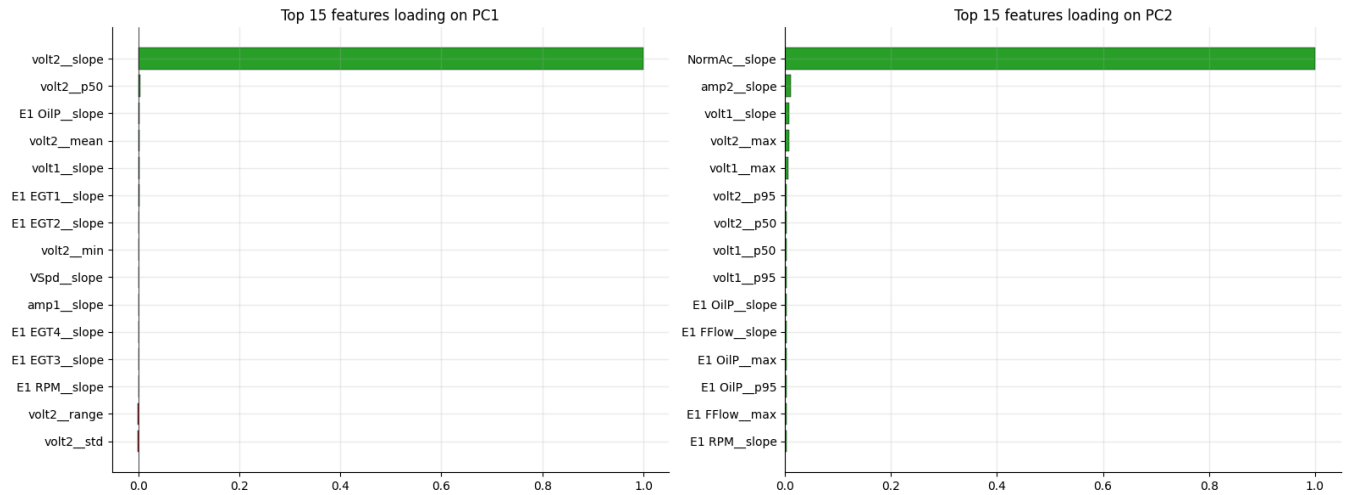
```

loadings['abs_PC2'] = loadings['PC2'].abs()

fig, axes = plt.subplots(1, 2, figsize=(16, 6))
top_pc1 = loadings.nlargest(15, 'abs_PC1').sort_values('PC1')
axes[0].barh(top_pc1.index, top_pc1['PC1'],
              color=['#d62728' if v < 0 else '#2ca02c' for v in top_pc1['PC1']],
              edgecolor='black', linewidth=0.3)
axes[0].axvline(0, color='black', linewidth=0.5)
axes[0].set_title('Top 15 features loading on PC1')

top_pc2 = loadings.nlargest(15, 'abs_PC2').sort_values('PC2')
axes[1].barh(top_pc2.index, top_pc2['PC2'],
              color=['#d62728' if v < 0 else '#2ca02c' for v in top_pc2['PC2']],
              edgecolor='black', linewidth=0.3)
axes[1].axvline(0, color='black', linewidth=0.5)
axes[1].set_title('Top 15 features loading on PC2')
plt.tight_layout(); plt.show()

```



```

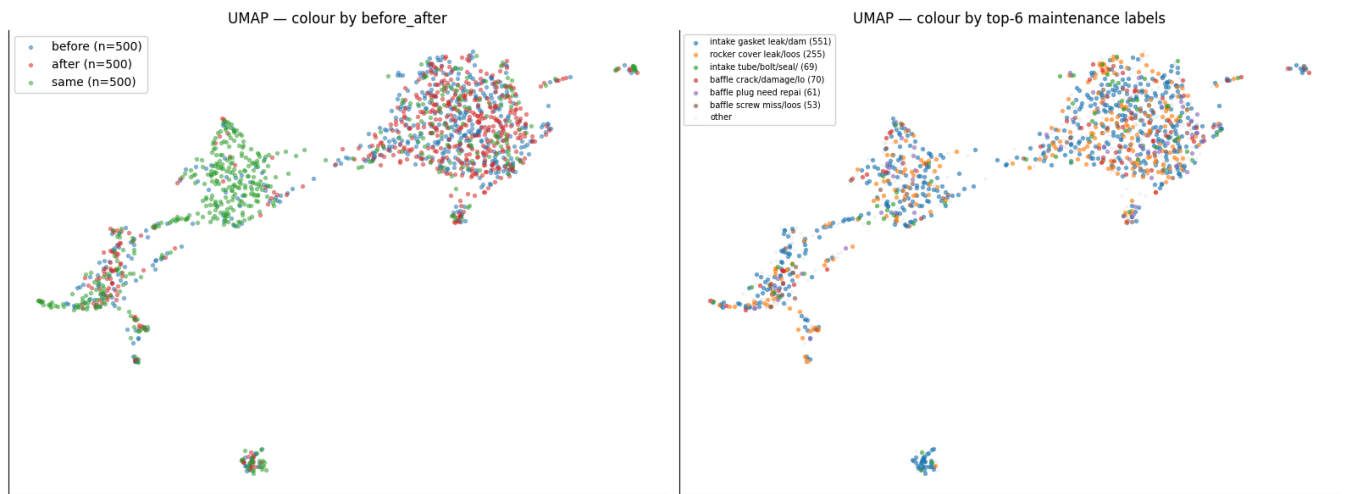
# UMAP (or t-SNE fallback) non-linear projection coloured by class
if HAS_UMAP:
    reducer = umap.UMAP(n_components=2, random_state=42, n_neighbors=30, min_dist=0.3)
    X_low = reducer.fit_transform(X_scaled)
    label_method = 'UMAP'
else:
    from sklearn.manifold import TSNE
    reducer = TSNE(n_components=2, random_state=42, perplexity=30, init='pca', learning_rate='auto')
    X_low = reducer.fit_transform(X_scaled)
    label_method = 't-SNE'

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# (a) Coloured by before_after
for cls, color in palette.items():
    mask = y_ba == cls
    if mask.any():
        axes[0].scatter(X_low[mask, 0], X_low[mask, 1], s=8, alpha=0.45,
                       color=color, label=f'{cls} (n={mask.sum()})')
axes[0].set_title(f'{label_method} - colour by before_after')
axes[0].legend()
axes[0].set_xticks([]); axes[0].set_yticks([])

# (b) Coloured by top-6 maintenance labels
top6_labels = label_counts.head(6).index.tolist()
cmap_labels = sns.color_palette('tab10', len(top6_labels))
for cls, color in zip(top6_labels, cmap_labels):
    mask = y_label == cls
    if mask.sum() > 0:
        axes[1].scatter(X_low[mask, 0], X_low[mask, 1], s=8, alpha=0.55,
                       color=color, label=f'{cls[:22]} ({mask.sum()})')
mask_other = ~np.isin(y_label, top6_labels)
axes[1].scatter(X_low[mask_other, 0], X_low[mask_other, 1], s=4, alpha=0.15,
               color='lightgray', label='other')
axes[1].set_title(f'{label_method} - colour by top-6 maintenance labels')
axes[1].legend(fontsize=7, loc='best')
axes[1].set_xticks([]); axes[1].set_yticks([])
plt.tight_layout(); plt.show()

```



```
# Mutual information – which features carry the most info about before_after?
mi_scores = mutual_info_classif(X_scaled, y_ba, random_state=42)
mi_df = pd.DataFrame({'feature': X.columns, 'mutual_info': mi_scores}).sort_values('mutual_info', ascending=False)

fig, ax = plt.subplots(figsize=(10, 7))
top_mi = mi_df.head(20).iloc[::-1]
ax.barh(top_mi['feature'], top_mi['mutual_info'], color='teal', edgecolor='black', linewidth=0.3)
ax.set_xlabel('Mutual information with before_after target')
ax.set_title('Top 20 features by mutual information')
plt.tight_layout(); plt.show()
print('\nTop 10 features by mutual information:')
mi_df.head(10).round(4)
```